

## Abstract

Post-quantum cryptography standards have been published, but organisations face a practical question with no empirical answer in the existing literature: what does it actually cost, in latency and storage, to integrate these algorithms into a complete cloud file-signing workflow rather than a standalone benchmark? This dissertation addresses that gap by implementing and benchmarking a security-hardened microservice prototype in Rust that models a realistic backend signing pipeline: API gateway, dedicated hashing service, manifest builder, PostgreSQL, and MinIO object store.

A formal Phase 1 campaign executes 63 000 measured runs across 630 conditions - seven signature profiles (RSA-PSS, ECDSA, EdDSA, HMAC-SHA256, ML-DSA, SLH-DSA, FN-DSA), three hash algorithms (SHA-256, BLAKE3, Keccak-256), five payload sizes (10 KB–50 MB), and six workflow scenarios - to produce decision-grade performance data at 95% bootstrap confidence. The central finding is that FN-DSA (Falcon) and ML-DSA (Dilithium), both NIST-standardised post-quantum signature schemes, impose *less* end-to-end latency than RSA-PSS across all payload sizes; FN-DSA in particular achieves faster signing than ECDSA and the fastest public-key verification in the study at 0.25 ms. SLH-DSA’s 1.6-second signing cost renders it unsuitable for any interactive cloud signing path. Hash algorithm choice is a secondary concern at small files; BLAKE3 outperforms SHA-256 by 7% at 50 MB while Keccak-256 trails both by 35% with no compensating advantage.

These results directly inform the Phase 2 hybrid benchmarking campaign - pairing the selected classical and post-quantum profiles in dual-signing configurations to model the transitional strategy recommended by NIST migration guidance - which is the subject of ongoing work. The dissertation contributes an open, reproducible benchmarking harness and a dataset of 63000 system-level measurements that practitioners can use to inform PQC migration decisions for cloud file-handling infrastructure.

# Contents

|          |                                                                        |           |
|----------|------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                                    | <b>7</b>  |
| 1.1      | Problem Statement . . . . .                                            | 7         |
| 1.2      | Research Questions . . . . .                                           | 8         |
| 1.3      | Aims and Objectives . . . . .                                          | 8         |
| 1.4      | Scope and Limitations . . . . .                                        | 9         |
| 1.5      | Report Structure . . . . .                                             | 10        |
| <b>2</b> | <b>Literature Review</b>                                               | <b>11</b> |
| 2.1      | Introduction . . . . .                                                 | 11        |
| 2.2      | The Quantum Threat to Classical Cryptography . . . . .                 | 11        |
| 2.3      | Harvest Now, Decrypt Later Risk . . . . .                              | 12        |
| 2.4      | Current State of the Cryptographic Standards . . . . .                 | 12        |
| 2.5      | The NIST Post-Quantum Cryptography Standardisation Programme . . . . . | 13        |
| 2.6      | Post-Quantum Algorithm Families . . . . .                              | 14        |
| 2.6.1    | Lattice-Based Cryptography: ML-KEM and ML-DSA . . . . .                | 14        |
| 2.6.2    | Hash-Based Signatures: SLH-DSA . . . . .                               | 15        |
| 2.6.3    | NTRU-Based Signatures: Falcon and FN-DSA . . . . .                     | 15        |
| 2.7      | Secure File Handling in Cloud Workflows . . . . .                      | 16        |
| 2.8      | Summary and Research Gaps . . . . .                                    | 17        |
| <b>3</b> | <b>Methodology</b>                                                     | <b>19</b> |
| 3.1      | Research Design and Approach . . . . .                                 | 19        |
| 3.2      | Workflow Exploration and Baseline Development . . . . .                | 20        |
| 3.3      | Storage Backend Selection . . . . .                                    | 21        |
| 3.4      | Manifest Design and Determinism . . . . .                              | 22        |
| 3.5      | Microservice Architecture Decision . . . . .                           | 22        |
| 3.6      | User Interface Decisions . . . . .                                     | 23        |
| 3.7      | Verification Framework Design . . . . .                                | 24        |
| 3.8      | Security Hardening . . . . .                                           | 25        |
| 3.9      | Algorithm Selection and Coverage . . . . .                             | 27        |
| 3.10     | Benchmarking Framework Design . . . . .                                | 28        |
| 3.11     | Dataset Design . . . . .                                               | 30        |
| 3.12     | Telemetry and Metrics Collection . . . . .                             | 30        |
| 3.13     | Statistical Analysis Methodology . . . . .                             | 31        |
| <b>4</b> | <b>Experimental Results and Analysis</b>                               | <b>33</b> |
| 4.1      | Campaign Provenance and Quality . . . . .                              | 34        |
| 4.2      | Hash Algorithm Performance . . . . .                                   | 35        |

|          |                                                        |           |
|----------|--------------------------------------------------------|-----------|
| 4.3      | Signing Performance . . . . .                          | 36        |
| 4.4      | Verification Performance . . . . .                     | 38        |
| 4.5      | Artifact Overhead . . . . .                            | 39        |
| 4.6      | End-to-End Workflow Latency . . . . .                  | 41        |
| 4.7      | Cloud-Workflow Contextualisation . . . . .             | 43        |
| 4.8      | Phase 2 Carry-Forward Selection . . . . .              | 45        |
| 4.9      | Threats to Validity . . . . .                          | 47        |
| 4.10     | Summary . . . . .                                      | 49        |
| <b>5</b> | <b>Discussion and Conclusion</b>                       | <b>50</b> |
| 5.1      | Answering the Research Questions . . . . .             | 50        |
| 5.2      | Synthesis: What This Means for Practitioners . . . . . | 52        |
| 5.3      | Contributions . . . . .                                | 53        |
| 5.4      | Limitations . . . . .                                  | 53        |
| 5.5      | Future Work . . . . .                                  | 54        |
| 5.6      | Concluding Remarks . . . . .                           | 55        |
|          | <b>Appendices</b>                                      | <b>61</b> |
| <b>A</b> | <b>Supplementary Submission Material</b>               | <b>61</b> |

## List of Tables

|   |                                                                                                                                                                                                                                       |    |
|---|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 1 | Security requirements for quantum-safe cloud-based file work-<br>flows . . . . .                                                                                                                                                      | 17 |
| 2 | Formal Phase 1 campaign summary. . . . .                                                                                                                                                                                              | 34 |
| 3 | Hash stage latency and throughput ( <b>sign_only</b> scenario, warm,<br>median). All seven signature profiles share an identical hash<br>stage; values are per-hash medians across profiles. . . . .                                  | 35 |
| 4 | Server-side signing latency by profile (SHA-256 hash, <b>sign_only</b><br>scenario, warm, median ms). Sign cost is computed on the 32-<br>byte digest and is therefore payload-independent; the range<br>shown confirms this. . . . . | 37 |
| 5 | Server-side verification latency by profile (SHA-256 hash, <b>verify_manifest</b><br>scenario, warm, median ms). Values are stable across payload<br>sizes; 10 KB and 10 MB columns are shown as representatives. . . . .             | 39 |
| 6 | Signature sizes and overhead (SHA-256, <b>sign_only</b> scenario,<br>warm, median). Signature-as-%-of-file values illustrate the<br>storage-amplification penalty at each bucket size. . . . .                                        | 40 |
| 7 | Client-observed end-to-end workflow latency (SHA-256, <b>workflow</b><br>scenario, warm, median ms). . . . .                                                                                                                          | 42 |
| 8 | Ratio of each profile’s end-to-end median to RSA-PSS median<br>(SHA-256, <b>workflow</b> scenario, warm). Values below 1.0 indi-<br>cate faster than RSA-PSS. 95% bootstrapped CI shown. . . . .                                      | 43 |
| 9 | Phase 2 carry-forward selection summary. . . . .                                                                                                                                                                                      | 47 |

## List of Figures

- 1 Server-side hash latency by algorithm across payload sizes (`sign_only` scenario, warm, median with 95% CI bands). BLAKE3 and SHA-256 are nearly indistinguishable at small sizes; Keccak-256 diverges progressively beyond 100 KB. . . . . 36
- 2 Server-side signing latency by profile across payload sizes (SHA-256 hash, `sign_only` scenario, warm, median). Note the logarithmic scale: SLH-DSA is separated from all other profiles by three orders of magnitude. All remaining profiles cluster between 0.18 ms and 5.3 ms. . . . . 38
- 3 Server-side verification latency by profile (SHA-256 hash, `verify_manifest` scenario, warm, median with 95% CI bands). FN-DSA achieves the fastest public-key verification in the suite. . . . . 40
- 4 Signature size by algorithm on a linear scale (median bytes per condition). The PQC schemes span two orders of magnitude, from FN-DSA’s 654 bytes to SLH-DSA’s 7 854 bytes. . . . . 41
- 5 Storage amplification ratio (total stored bytes / raw file bytes) by profile and payload size. PQC overhead is most pronounced for small payloads; FN-DSA remains competitive with RSA-PSS across the full size range. . . . . 42
- 6 End-to-end throughput (MiB/s) by profile across payload sizes. All non-SLH-DSA profiles produce nearly identical throughput curves; SLH-DSA is separated by two to three orders of magnitude at small sizes. . . . . 44
- 7 Server-side overhead ratio relative to RSA-PSS baseline with bootstrapped 95% confidence intervals (`sign_only` scenario). Ratios below 1.0 indicate lower overhead than RSA-PSS. FN-DSA, ML-DSA, EdDSA, and ECDSA are all below the RSA-PSS reference. . . . . 45

## Acknowledgements

I would like to thank my supervisor, peers, friends, and family for their support throughout this project.

Their feedback, patience, and encouragement helped shape both the implementation and the dissertation.

# 1 Introduction

Modern digital systems rely heavily on cryptographic algorithms to preserve the authenticity and integrity of files. Standardised security mechanisms, such as Rivest-Shamir-Adleman (RSA) and the Elliptic Curve Digital Signature Algorithm (ECDSA), rely on the computational intractability of integer factorisation and the discrete logarithm problem. However, the long-anticipated emergence of reasonably powerful quantum computers poses a significant and inevitable risk to the privacy and integrity of data protected by widely adopted classical public-key cryptography. Although these computational problems are challenging to overcome using traditional methods, quantum algorithms have demonstrated that both can be solved in polynomial time using Shor’s algorithm [48], rendering current public-key schemes insecure against a sufficiently capable quantum adversary. Cloud environments are particularly vulnerable due to the frequent exchange of sensitive data across untrusted networks and their reliance on persistent key material.

In response, the National Institute of Standards and Technology (NIST) initiated the Post-Quantum Cryptography (PQC) standardisation programme and, in August 2024, published the first four quantum-resistant standards: FIPS 203 (ML-KEM), FIPS 204 (ML-DSA), FIPS 205 (SLH-DSA), and FIPS 206 (FN-DSA) [36–39]. Integration of these schemes into file-handling cloud workflows requires a detailed, empirically grounded understanding of their performance characteristics. This dissertation addresses that need by implementing and benchmarking a prototype system across a matrix of signature and hash algorithm combinations.

## 1.1 Problem Statement

Existing evaluations of quantum-resistant algorithms are commonly conducted in isolation, measuring raw cryptographic performance without accounting for the broader system context. Even where system-level benchmarking exists, a limited number of studies have used cloud-like file-handling workflows as their base. These evaluations typically omit the key aspects of real-world production deployments: interaction with object storage, end-to-end request handling across service boundaries, manifest generation and signing, and multi-step integrity verification.

As a result, empirical understanding of the behaviour of quantum-resistant schemes within complete workflows remains limited. Organisations considering migration need to understand the practical trade-offs of relying solely on classical algorithms, adopting PQC-only configurations, or employing hybrid approaches in which classical and post-quantum mechanisms operate in com-

bination. As system pipelines have limited bandwidth and associated cost, the most balanced solution needs to be identified through evidence rather than assumption.

Furthermore, hybrid approaches - where classical and post-quantum mechanisms are combined as a transitional measure while novel technologies are still undergoing field testing - remain insufficiently explored to support meaningful recommendations for cloud-based signing infrastructure.

## 1.2 Research Questions

This dissertation is guided by the following research questions:

- RQ1** What is the performance overhead of post-quantum and hybrid signature schemes compared to classical signatures in a complete end-to-end file-processing lifecycle?
- RQ2** How does hybrid signing affect latency and system throughput at matching bandwidth relative to classical-only and PQC-only strategies?
- RQ3** What architectural harnesses are required to enable reproducible measurement and analysis of cryptographic signature schemes?
- RQ4** What are the security implications and trade-offs of integrating PQC and hybrid signatures into an existing backend microservice architecture?

## 1.3 Aims and Objectives

The aim of this project is to implement a prototype of a security-hardened, backend-centric system that demonstrates file signing and verification workflows using a variety of hashing and signature techniques, and to use this prototype to explore the efficiency and feasibility of PQC-only and hybrid approaches relative to classical baselines. While a web user interface is included for demonstration purposes, all benchmarking is conducted via a Command-Line Interface (CLI) to prevent performance penalties associated with browser operations. For each assessment campaign, tests are conducted in a controlled and reproducible randomised sequence of file-processing workflow runs.

To achieve this, the project sets the following objectives:

1. Design a system architecture that enforces consistent cryptographic execution and minimises client-side variability.



2. Implement support for a variety of classical, post-quantum, and hybrid signature and hash profiles over a shared data model. Selected algorithms span classical schemes and all three NIST-standardised post-quantum signature families (ML-DSA, SLH-DSA, FN-DSA), providing representative coverage of the current cryptographic landscape.
3. Develop a structure for canonical manifest generation to guarantee deterministic and reproducible outputs.
4. Integrate mechanisms for secure file hashing, signing, storage, and verification. For the verification flow, develop a framework for checking file integrity beyond mere hash matching.
5. Construct a benchmarking framework that generates reproducible performance measurements across multiple configurations, with randomised run ordering seeded for deterministic sequences.

## 1.4 Scope and Limitations

This dissertation is scoped to the signing and verification phases of a backend cloud file workflow. The prototype implements a microservice architecture comprising an API gateway, a dedicated hashing service, and a manifest-building and signing service, backed by a PostgreSQL database and a MinIO object store deployed in a containerised local environment. Benchmarking is conducted in two phases. The first phase covers seven signature profiles across two categories - classical (RSA-PSS, ECDSA-P256, EdDSA, HMAC-SHA256) and post-quantum (ML-DSA/FIPS 204, SLH-DSA/FIPS 205, FN-DSA/FIPS 206) - evaluated against three hash algorithms (SHA-256, BLAKE3, Keccak-256) to identify the two best-performing representatives from each category. The second phase uses those selected profiles in hybrid classical-plus-PQC dual-signing configurations. Key encapsulation, transport security, and client-side encryption are explicitly out of scope.

Several limitations apply. The benchmarking environment is a single-host Docker deployment and does not replicate the network topology, storage I/O characteristics, or managed-service latencies of a live public cloud environment. Hybrid evaluation is limited to the classical-PQC pairings selected by the first-phase performance analysis; other combinations are not covered. Only representative parameter sets are selected for each algorithm. Formal security analysis, side-channel resistance, and fault-injection tolerance are not addressed. Execution timings are subject to OS scheduler variability; this is mitigated through warm-up exclusion and randomised run ordering but cannot be entirely eliminated.

## 1.5 Report Structure

The remainder of this dissertation is organised as follows. Section 2 presents the literature review, covering the quantum threat landscape, the HNDL risk model, limitations of current cryptographic standards, the NIST PQC standardisation programme, post-quantum algorithm families and their security foundations, and cloud file workflow security requirements. Section 3 describes the methodology: the research design, all significant iterative development decisions from proof-of-concept to formal campaign, security hardening, and the complete benchmarking and statistical analysis framework. This section is structured as a narrative of the design process rather than a static description, ensuring that the rationale behind each architectural choice is traceable. Section 4 presents the experimental results and analysis of the formal Phase 1 benchmarking campaign, reporting hash, signing, verification, and end-to-end workflow measurements and deriving the Phase 2 carry-forward selection. Section 5 synthesises the findings, discusses their implications for organisations considering PQC migration, reflects on limitations and threats to validity, and identifies directions for future work.

## 2 Literature Review

### 2.1 Introduction

This section reviews the body of literature relevant to post-quantum cryptography in the context of cloud-based file-handling workflows. It begins by establishing the quantum threat to classical public-key cryptography and the specific risks this poses for cloud storage. It then surveys the NIST standardisation process and the principal post-quantum signature algorithm families selected for standardisation. Finally, it examines the security requirements of cloud file workflows and identifies the research gaps that motivate this dissertation.

### 2.2 The Quantum Threat to Classical Cryptography

Classical public-key cryptography is a foundation of multiple functions in cloud workflows for file handling: secure key establishment, authentication and non-repudiation, and key wrapping for envelope encryption. Current industry-standardised schemes, such as RSA and elliptic-curve cryptography, are powerful enough to withstand brute-force attacks from traditional computers [35]. However, this is not the case for emerging quantum computers. Shor’s algorithm [48], if implemented on a large enough quantum computer - estimated in 2021 to require around 20 million noisy physical qubits [12] - would render such schemes insecure. The timescale for potential factorisation of RSA-2048 was estimated to be approximately 8 hours under those quantum resource assumptions [12]. Grover’s algorithm [15], on the other hand, reduces the effective security of symmetric key encryption primitives like AES and SHA-2 by providing a quadratic speedup for key search, effectively halving their security level, although it does not entirely eliminate their protection [6, 15]. Reinforcing workflows to become quantum-safe must therefore prioritise replacement of asymmetric cryptography, while symmetric and hash-based primitives can be retained at appropriately increased security parameters.

Beyond the capabilities of quantum hardware itself, the urgency of migration depends on the intersection of two timelines. Mosca [27] formalised this risk: an organisation faces a cryptographic threat today if the sum of the time needed to migrate its infrastructure and the time until a cryptographically relevant quantum computer arrives exceeds the required security shelf life of its protected data. The engineering challenges that separate current noisy intermediate-scale quantum (NISQ) devices [44] from fault-tolerant systems capable of executing Shor’s algorithm at cryptographic scale remain signifi-

cant; however, the pace of hardware development has led governments and standards bodies to treat this as a concrete planning horizon.

### **2.3 Harvest Now, Decrypt Later Risk**

The "Harvest Now, Decrypt Later" (HN DL) threat model is exceptionally severe for cloud storage because the data may be hijacked mid-transport from an intercepted TLS connection to object stores. Moreover, it may be exfiltrated from the compromised data warehouses, allowing an attacker to steal entire objects. This data may have limited intelligence value today, but the malicious actors can store both the ciphertext and its public key. Once a decent quantum computer becomes available, deriving private keys and decrypting stored data retroactively [16, 46] becomes a matter of hours. This is a genuine security and compliance concern for organisations that are required to retain certain data for long periods, such as regulated financial records or intellectual property.

The implications extend directly to signing infrastructure: if long-lived signing keys are exposed today, a future adversary with access to sufficient quantum computing could retroactively forge signatures on historical manifests, undermining the non-repudiation guarantees on which auditable records depend. Organisations retaining digitally signed file manifests over periods that may extend into the anticipated quantum era should treat this as an immediate strategic risk.

### **2.4 Current State of the Cryptographic Standards**

Cryptographic standards specify which algorithms may be used to generate a digital signature. Current standards like FIPS 186-5 [35], despite realistic quantum-related security concerns, still rely on RSA and ECDSA. These schemes offer barely any resistance against quantum attacks - Shor's algorithm can efficiently break both. This limitation also applies to most existing FIPS 140-2 and FIPS 140-3 certified hardware security modules (HSMs) and key management services (KMS), although PQC support is appearing more frequently in select vendor firmware [14]. Major cloud providers have begun deploying PQC features in their services: AWS implemented ML-KEM hybrid post-quantum TLS in AWS KMS, ACM, and Secrets Manager as of April 2025 [47]. Moreover, Google Cloud added ML-DSA and SLH-DSA signature support to Cloud KMS in February 2025 [9]. Given that the leading providers have made such contributions to their core infrastructure services, this indicates that PQC is not merely viable but is likely to see widespread adoption in the near future.

The X.509 certificate infrastructure also remains in transition. The IETF LAMPS working group is actively developing PQC certificate specifications, including algorithm identifier formats and certificate profile definitions [22, 23]. Despite meaningful progress, widespread production deployment of PQC-signed certificates by commercial CAs remains limited as of Q2 2026, with the ecosystem still dominated by readiness work and early commercial product support rather than broad WebPKI rollout [? ].

## 2.5 The NIST Post-Quantum Cryptography Standardisation Programme

In December 2016, NIST announced a public competition for the standardisation of PQC algorithms and invited submissions [31]. There were 82 candidate algorithms proposed across lattice-based, code-based, multivariate and hash-based families [32, 41]. After a year-long evaluation, only 26 algorithms passed to the second round in January 2019 [32], with further analysis leading to 7 finalists and 8 alternative candidates announced in July 2020 [33]. The next two years of theoretical and empirical analysis allowed NIST to publish the first batch of post-quantum standards in August 2024 [36–39]:

- **CRYSTALS-Kyber (ML-KEM)** - standardised as FIPS 203 [36]
- **CRYSTALS-Dilithium (ML-DSA)** - standardised as FIPS 204 [37]
- **SPHINCS+ (SLH-DSA)** - standardised as FIPS 205 [38]
- **Falcon (FN-DSA)** - in development as FIPS 206 [39]

CRYSTALS-Kyber is a Key Encapsulation Mechanism (KEM) used to establish a shared secret over an insecure network. ML-DSA, SLH-DSA, and FN-DSA are digital signature schemes providing authentication and non-repudiation. The transition guidance was published along with the standards [30, 40] and the U.S. National Security Memorandum NSM-10 have set deadlines for federal systems and high-risk sectors to transitions to quantum-safe infrastructure. A forth round of evaluation allowed NIST to add HQC, a code-based alternative KEM, as a fifth algorithm for PQC encryption in March 2025 [42, 43], while SIKE, which had been a candidate, was broken and withdrawn [8].

## 2.6 Post-Quantum Algorithm Families

The three standardised signature algorithms span two distinct design families: lattice-based and hash-based. Since they have different security foundations and performance characteristics, the suitability varies for deployment in bandwidth-sensitive or constrained environments. The following subsections survey each family.

### 2.6.1 Lattice-Based Cryptography: ML-KEM and ML-DSA

Lattice-based cryptography underpins ML-KEM and ML-DSA, the two lattice-based NIST standards. Their security relies on the computational hardness of the Module Learning with Errors (MLWE) problem and related challenges, including the Shortest Vector Problem (SVP) and Closest Vector Problem (CVP) [36, 37, 45]. These problems are connected through worst-case-to-average-case reductions, which means that in the case of a randomly chosen average-case MLWE instance being solved, it would mean the ability to solve the hardest worst-case lattice problems. [45]. ML-DSA algorithm builds on the Fiat-Shamir with Aborts framework introduced by Lyubashevsky [21], yielding an efficient signature scheme without requiring a computationally expensive tools to generate secret values.

Lattice-based schemes offer three key advantages for deployment:

**Strong Security Reductions** A key distinguishing feature of lattice-based PQC schemes over classical alternatives is the nature of their security reductions. ML-KEM and ML-DSA possess strong worst-case-to-average-case reductions [36]: an adversary capable of breaking these schemes on randomly chosen average-case instances would necessarily be capable of solving the hardest known worst-case instances of SVP and CVP. Conversely, classical schemes like RSA and ECDH rely on unproven average-case hardness assumptions without equivalent worst-case reductions, potentially making them more susceptible to unexpected algorithmic breakthroughs [36, 37]. This mathematical rigour provides a stronger theoretical foundation for long-term trust.

**Implementation Flexibility** ML-KEM and ML-DSA run efficiently across CPUs, HPC servers, embedded devices and cryptographic accelerators.

**Resistance Against Quantum Cryptanalysis** Recent analyses by NIST and the research community confirm that the assumptions behind ML-KEM and ML-DSA remain unbroken [36, 37]. There is no known quantum algorithm posing a threat to schemes at their specified security levels.

### 2.6.2 Hash-Based Signatures: SLH-DSA

SLH-DSA (FIPS 205 [38]) relies on the collision and pre-image resistances of the underlying hash functions (SHA-256 or SHAKE-256, depending on the variant), with no dependence on algebraic hardness assumptions. This approach traces its lineage to the one-time signatures of Lamport [20] and the tree-based multi-signing construction of Merkle [25]. A key feature that defines SLH-DSA is stateless design, which differentiates it from earlier stateful hash-based schemes like XMSS [18] or LMS [24]. Stateful schemes require secure management and synchronisation of signing state to prevent key reuse [34] - a significant operational burden in distributed cloud environments, which SLH-DSA eliminates entirely.

The trade-off for this conservative security model is signature size. SLH-DSA-SHA2-128s (smallest variant) produces signatures of 7856 bytes, compared to ML-DSA-44, which has higher security category, that is only 2420 bytes [37, 38]. This introduces non-trivial overhead in manifest pipelines and bandwidth-sensitive interfaces. In some cases, signing throughput is also lower than lattice-based alternatives, which must be weighed against the stronger security assurances the scheme provides.

### 2.6.3 NTRU-Based Signatures: Falcon and FN-DSA

Falcon [10] is a lattice-based signature scheme constructed over NTRU lattices - a family introduced by Hoffstein, Pipher, and Silverman [17]. Unlike ML-DSA with its rejection sampling, Falcon employs a Gaussian lattice trap-door sampler over NTRU lattices, achieving remarkably light output: Falcon-512 produces signatures of 666 bytes compared to 2420 bytes for ML-DSA-44 [37]. Being in late-stage development and awaiting final standardisation as FN-DSA in FIPS 206 [39], Falcon completes the initial NIST post-quantum signature portfolio.

However, the compactness comes with implementation complexity: the Fast Fourier Transform-based Gaussian sampler is numerically sensitive, therefore has to be precisely adjusted for floating-point handling to have constant-time operation. This makes Falcon noticeably more demanding than ML-DSA to implement securely. In contexts where bandwidth is a primary bottleneck, such as high-throughput pipelines or API gateways, Falcon's smaller signatures may provide a performance advantage that is more noticeable than in isolated benchmarks.

## 2.7 Secure File Handling in Cloud Workflows

A typical cloud file workflow is built around multiple phases where each requires integrity verification. The pattern usually uses an envelope encryption architecture - files are first encrypted locally using a data encryption key (DEK). The DEK itself is then protected by a key encryption key (KEK), and cloud key management service (KMS) stored and manages it. Thus, the file is protected by the DEK, and the DEK is protected by the KEK [5, 13].

The workflow begins with the upload phase. Client generates or fetches the DEK and encrypts the file locally, then the produced ciphertext and DEK encrypted by KEK are uploaded to object storage (like AWS S3). In this case, DEK is typically embedded in metadata or cached in a separate key store [2, 49].

Then comes the storage phase. The service computes and stores checksums to guarantee that data remains unchanged during persistent storage. Modern cloud platforms have support for a long list of checksum algorithms, so these can be used to verify files against silent data corruption or transmission errors [3, 4].

To be able to trace the activity, all object access is recorded to an audit log. Log and manifest files may be signed with digital signatures to support non-repudiation and tamper-evidence. This allows organisations to comply with data storage regulations and detect unauthorised data modification [1].

In the verification phase, the client retrieves the encrypted object and its wrapped DEK. After providing credentials to the KMS to unwrap the DEK, the files are locally decrypted to recompute the checksum and verify their integrity by comparing it to the stored value. Any mismatch should raise an integrity violation alert [11]. All interactions are logged with timestamps, identity, action, and resource affected - this maintains a chain of custody. In some high-risk environments the logs themselves are signed to prevent manipulation [19].

Table 1 outlines the core security requirements for quantum-safe cloud file workflows, the mechanisms used to meet the requirements, and their implications for post-quantum deployment.



Table 1: Security requirements for quantum-safe cloud-based file workflows

| Requirement             | Mechanism                       | PQC Implications            |
|-------------------------|---------------------------------|-----------------------------|
| <b>Confidentiality</b>  | DEK wrapped by KEK              | KEK derivation using ML-KEM |
| <b>Integrity</b>        | Checksums, manifest signatures  | ML-DSA/SLH-DSA signatures   |
| <b>Authenticity</b>     | Signatures on metadata and logs | Mandatory PQC signatures    |
| <b>Non-repudiation</b>  | Digital signatures              | ML-DSA/SLH-DSA signatures   |
| <b>Key Management</b>   | HSM/KMS stored keys             | KMS with PQC support        |
| <b>Auditability</b>     | Immutable signed logs           | Logs signed with PQC        |
| <b>Chain of Custody</b> | Custody record signatures       | Timestamps, PQC signatures  |

## 2.8 Summary and Research Gaps

The literature reviewed in this section confirms that the quantum threat to classical public-key cryptography standardised nowadays is credible and time-sensitive. The HNDL threat model makes eventual compromise an immediate concern for cloud-stored and signed data. NIST has developed a robust portfolio of standards for post-quantum signatures across two distinct design families with one more in final stage development. Yet, the research gaps still remain.

First, despite there a wide variety of isolated algorithm benchmarks being available, system-level studies of PQC signature schemes specifically in complete file-handling workflows are limited. Here the emphasis lies on upload, hashing, manifest signing, object storage interaction, and verification. Most published evaluations focus on TLS handshakes or standalone microbenchmarks that do not reflect the network latency that the workflows have.

Second, no published work provides a controlled empirical comparison of classical-only, PQC-only, and hybrid signing strategies within a unified microservice architecture under consistent experimental conditions, which makes it difficult for organisations to determine the real cost and assess performance trade-offs of different migration strategies.

Third, the relative performance characteristics of the three NIST-standardised PQC signature families (lattice-based ML-DSA, hash-based SLH-DSA, and NTRU-lattice-based FN-DSA/Falcon) have not been compared at the sys-

tem level within a complete file-signing workflow. This report provides not only full signing and verifying workflows, but also specific partial ones.

Fourth, the harnesses that are required to generate reproducible, statistically valid benchmarks of cryptographic schemes in such pipelines are still undocumented.

These gaps directly motivate the research questions set out in Section 1.2 and form the basis of the prototype design and benchmarking methodology described in the subsequent sections.

## 3 Methodology

This section describes the research methodology used for design, implementation and evaluation of the prototype system. The project followed a continuous iterative approach of building a prototype, where decisions were refined incrementally at each point where components were build and the architectural suitability assessed. This section traces the development timeline starting from initial study of cloud file-handling workflows and all the way to the construction of a benchmarking framework. Methodology also explains the rationale behind each major architectural and experimental decision with the trade-off analysis. This directly addresses RQ3 - while providing the context to interpret the empirical results reported in Section 6.

### 3.1 Research Design and Approach

The research follows a prototype-driven empirical methodology, prioritising construction of a representative system rather than measuring cryptographic primitives in isolation. The aim was to and measure performance across complete signing and verification workflows: service communication, storage I/O, manifest serialisation and database persistence. This approach was chosen because of research focus on practical results of system-level latency testing that cannot be reliably produced by standalone microbenchmarks.

Development was executed around an iterative cycle: explore an area, plan an implementation strategy, perform trade-off analysis, select an approach, develop a component, conduct tests, identify limitations, and refine. This loop was repeated across each layer. A local containerised environment (Docker) was used throughout in preference to a live cloud deployment for three main reasons.

First, a local environment eliminates amplified variability introduced by network topology and per-request pricing, which measures performance of algorithmic and architectural differences, not just infrastructure noise.

Second, it removes any recurring cloud API and object storage cost at the scale required for a formal benchmarking campaign. Despite a cost estimation for each campaign sitting around £6, the testing required far more than a single run.

Third, Docker-based containerised deployment ensures that every user can reproduce the system configuration from the repository without manual installataion of external dependencies.

Rust was selected as the implementation language for all backend services. Its zero-cost abstractions and lack of a garbage collector make it well-suited to latency-sensitive measurement work, since pauses from runtime garbage col-

lection would skew timing data. Rust’s type system and built-in compile time memory-safety guarantees also reduce the risk of undefined behaviour when developing a concurrent systems with multiple services. It also has mature PQC crate ecosystem, including all of the signature schemes required for this project - `fips204` (ML-DSA), `fips205` (SLH-DSA), and `pqcrypto-falcon` (Falcon/FN-DSA).

### 3.2 Workflow Exploration and Baseline Development

The first step was to research how the actual cloud file-handling workflows function in production. Specifically, the focus was on the phases described in the literature review: upload, checksum verification, signing, storage, and download with integrity checking. This reading established the understanding of a prototype architecture that would need to be built.

To deepen the understanding, the documentation of the major cloud object storage platforms was reviewed, including AWS S3 object integrity documentation [3, 4], the AWS KMS server-side and client-side encryption guidance [2, 5], the Google Cloud KMS and Yandex Cloud KMS envelope encryption references [13, 49], and the AWS CloudTrail log-file integrity validation specification [1]. Additionally, the source-code of `aws-s3-integrity-check` tool [11] was examined to gain knowledge about existing integrity checkers. MinIO’s documentation [26] was reviewed as self-hosted S3-compatible backend, confirming that the same client code and APIs would transfer to a real cloud deployment. Yet, since the beginning of the work on this project, MinIO maintainers have abandoned open-source support in favour of enterprise-only [? ]

The analysis confirmed that the reference architecture across all platforms is consistent. To begin with, files are encrypted locally under a per-object data encryption key (DEK), which is wrapped by a key encryption key (KEK) stored in a cloud KMS - example of an envelope encryption pattern [5, 13]. The encrypted object and its wrapped DEK are uploaded to object storage. The platform then calculates and stores a checksum at the time of ingestion [2, 3]. When the objects are retrieved, the stored checksum is recomputed and matched against the stored value. If the strings differ, this may suggest silent data corruption or unauthorised modification [4, 11].

With this fundamental understanding, the development of a terminal-based proof of concept has started. A simple program was put together that took two arguments - hash function and local absolute file path. Hashing algorithms (SHA-256, BLAKE3, or Keccak-256 ) were executed locally to produce hex-encoded string output for file specified in one of the arguments. Signature operations were then introduced: user specified the third argu-

ment for signing primitive, which ingested the file bytes, producing a (digest, signature) pair in the terminal.

At this point, there was an early design choice that was required to proceed. The initial prototype signed the raw file content, reflecting the simplest possible implementation. However, two considerations motivated a change to signing the file’s hash instead: performance and security.

First, the hash acts as a fixed-size deterministic string produced from a file: no matter the file size, the signing primitive works on a digest of the same length. Therefore, there is no need to re-read the full file during verification, which reduces latency for larger payloads.

Second, this prevents length-extension attacks and partial-file presentation attacks, because signing operation is applied to a cryptographic digest rather than message with non-fixed length. These vulnerabilities are mitigated because any modifications to the file bytes changes the digest and therefore render the signature invalid.

This decision laid groundwork for adjustment of the workflow, so this hash-then-sign pattern was brought forward throughout the remainder of the prototype.

### 3.3 Storage Backend Selection

Once hash-then-sign was established, a need for a persistent storage appeared. The prototype is capable of one-off hashing and signing, but the objects required quick on-demand retrieval. The target was an S3-compatible object store to mirror the API surface used by production cloud deployments such as AWS S3 or Google Cloud Storage. This was the key to ensure that performance findings that would be transferable to real cloud environments.

AWS S3 itself was not used because continuous upload and download traffic across a benchmark campaign would incur substantial per-request and egress-bandwidth charges, which is not sustainable for a zero-budget research prototype. MinIO [26] was selected as the storage backend. It is open-source (public repository was maintained until February 2026), self-hosted, and exposes an S3-compatible API - good fit for the use cases where client code needs to target either MinIO or a real commercial S3 service with only an endpoint and credentials change. MinIO therefore was deployed as a Docker container alongside the other services.

There also was a need for a database that persistently stores manifest metadata and request-to-storage mapping. The two most prominent candidates were PostgreSQL and MySQL. While MySQL is great for simple web development, it would require many workarounds for the project’s data model, while PostgreSQL supports it naively. It was chosen as the relational

database for manifest metadata and request-to-storage mapping, as well as audit logs. ACID guarantees assure that a manifest is never partially written, plus its query interface supports the manifest-lookup operations required for the verification service.

### 3.4 Manifest Design and Determinism

To cryptographically guarantee a file’s integrity, a hash and a signature alone are insufficient. The full set of integrity and audit requirements identified in the literature review must be implemented. Moreover, there is no way of determining the hash and signature algorithms used, file size, date or object destination without additional metadata. Therefore, there was a need for a structured manifest record, which was introduced to carry all this along with the cryptographic material

The core requirement for the manifest is to be *deterministic*: given the matching file and the same algorithm identifiers, the manifest serialisation must produce an identical byte sequence every time in each independent run. This is crucial for verification flows as the service re-constructs the manifest from the uploaded file and compares its canonical representation against the one stored. Any non-determinism or even a single whitespace misalignment in serialisation would cause verification failures.

To achieve determinism requirements, canonicalisation was applied before serialisation. The manifest has a list of fields, which are sorted into a defined key order and the variable-length collections are then normalised to a canonical form before encoding. CBOR (Concise Binary Object Representation, RFC 8949 [7]) was selected for manifest serialisation because its deterministic encoding mode defines a total ordering for map keys and a canonical representation for all data types, guaranteeing that the same manifest always produces the same sequence of bytes. Besides that, CBOR also offers a smaller binary encoding than JSON, making it more relevant in large volumes of stored manifests and their sizes are included as an artifact metric in the benchmark.

### 3.5 Microservice Architecture Decision

As the number of components grew it became clear that a monolithic structure would not serve the measurement goals of the project. While having every service running in the same binary is a quicker way to iterate through development, if all operations were executed within a single process, it would be more challenging to attribute latency to specific stages, and the architecture would not resemble the distributed service topologies used in production

cloud environments.

The system was therefore decomposed into three independently deployable microservices:

**API Gateway** Receives client requests, enforces authentication and rate limiting. Routes requests to the corresponding downstream services. All client-side timing measurements are taken at this boundary.

**Hasher Service** Receives a file + an hashing algorithm id and returns the computed digest. Exposes a dedicated endpoint so that service latency can be independently measured.

**Manifest Builder Service** Constructs the canonical manifest, generates signature, stores the manifest in PostgreSQL, and stores the signed object in MinIO. Verification requests are also handled here, giving access to both stored manifests and the signing pipeline needed to re-derive manifests for comparison.

Each service lives in its dedicated Docker container. Nginx acts as a reverse proxy fronting the API gateway, and Docker Compose orchestrates the stack. The reasoning behind the use of microservices was not merely architectural convenience. Separating the functionality into dedicated services makes stage-level telemetry possible, which is crucial for evaluation. Each service does computing independently and reports the latency of its own operations. These decisions directly support RQ1 (identifying which stage contributes most to PQC overhead) and RQ3 (constructing a harness capable of producing reproducible, granular measurements).

### 3.6 User Interface Decisions

Three interface options were considered for interacting with the system.

A terminal user interface (TUI) was the first candidate, as it would keep all interaction within the terminal, which would enable quick navigation and configuration, as well as avoiding browser overhead. A TUI implementation was attempted but abandoned - the complexity of building a robust, navigable TUI in Rust was disproportionate to the benefit, since the interface was only needed for demonstration purposes rather than as a component under measurement. Besides, the navigation turned out to be significantly less intuitive, with would be amplified by the lack of exposure to TUI for non-technical users.

A React web UI was implemented instead with Vite frontend tool. It was built to provide users a browser-based interface through which they can

configure sign and verify flows, as well as inspect the results of these flows (view manifests, verification results). This interface serves its demonstration purpose and does not require any other special tooling beyond a browser. The web UI is not used for benchmarking, therefore its performance is irrelevant to the measurement results.

Benchmarking required a lightweight, low-latency solution that could automate performance measurements. Based on this need, a dedicated command-line interface (CLI) tool (`benchmark-cli`) was built. Browser-based testing was explicitly ruled out because it introduces variable latency from rendering and the browser's own network stack; such overheads are not under control in the testing environment and would contaminate timing measurements with noise. The CLI tool sends HTTP requests to the API gateway, measures latency using monotonic Rust `Instant` timers, and produces structured sequence of JSON records that can be deterministically replayed using a seed parameter. This design is also amenable to automated campaign scripting.

### 3.7 Verification Framework Design

The verification service was designed around the principle of *verification by re-execution*. This means that in a typical verification flow, the verifier uploads a file which undergoes the same sequence of computing operations (hashing, manifest creation, signing). These artifacts are then compared to the stored records retrieved by request ID. This reason why this approach is superior to hash values comparison is because it checks the full set of manifest fields, providing multi-dimensional integrity check.

The user submits two inputs to the verification endpoint: the file to be checked and the request ID of the original signing operation. The service fetches the stored manifest, re-derives the manifest for the submitted file, and evaluates a set of named verification checks:

- `file.provided.hash.match` — the re-derived hash matches the hash stored in the manifest.
- `storage.object.hash.match` — the object stored in MinIO hashes to the same digest as recorded.
- Signature validity checks, one per active signature scheme — the stored signature correctly verifies against the canonical signing bytes.
- Manifest field consistency checks — algorithm identifiers, file size, and CBOR envelope structure match between the re-derived and stored manifests.



Each check returns a `passed` flag and a `details` string. The overall result is PASS if and only if all checks pass, and FAIL otherwise. The check list was expanded iteratively: early versions of the verifier checked only hash match; additional signature and field checks were added after identifying that hash collision alone was insufficient to detect all classes of manifest substitution.

Six measurement scenarios were defined to decompose the end-to-end workflow into independently measurable stages:

**workflow** Complete end-to-end: upload file, hash, sign, store to MinIO, persist manifest to PostgreSQL, verify.

**sign\_only** Upload file and produce a signed manifest without any verification step.

**verify\_manifest** Compare manifest metadata only, without fetching the stored object from MinIO.

**verify\_stored** Verify the hash of the stored MinIO object against the manifest record.

**verify\_uploaded** Re-hash and re-sign the uploaded file and compare the resulting manifest against the stored record.

**verify\_full** Combine all verification checks: stored object hash verification plus uploaded content re-signing and manifest comparison.

Running each scenario as an independent condition in the benchmark matrix allows the cost of each verification stage to be isolated and compared.

### 3.8 Security Hardening

Before committing to the benchmarking phase, the prototype was reviewed against common vulnerability classes that appear in cloud-based file-handling systems. Literature on cloud security identified four classes of particular relevance to a file upload and processing service:

1. **Time-of-Check-Time-of-Use (TOCTOU)**. A symlinked path could be swapped between the validation step and the file-read step, causing the service to process a file it did not intend to. This was mitigated by traversing every path component under the allowed upload directory using `symlink_metadata` at each component, rejecting any path that contains a symbolic link. Enabling symlink following requires simultaneous activation of two distinct environment flags (`FOLLOW_SYMLINKS`

and `ALLOW_INSECURE_FOLLOW_SYMLINKS`), a double-opt-in that prevents accidental exposure.

2. **Null-Byte Path Injection.** Filenames containing a null byte (`\0`) can truncate at the OS level, causing the service to access a different path than intended. Any path containing a null byte is explicitly rejected before further processing.
3. **Directory Traversal.** Path components containing `..` could escape the upload directory and reach arbitrary filesystem locations. Component-level path walking rejects any path that attempts to traverse above the root of the allowed directory. An explicit whitelist (defaulting to `/data/uploads`) enforces that all file access is confined to the designated upload area.
4. **Sensitive File Exposure.** Unrestricted path access could expose system files under prefixes such as `/etc`, `/root`, `/proc`, and `/sys`. An 11-entry denylist of sensitive path prefixes is enforced in addition to the whitelist, providing defence in depth.

Additional hardening measures were applied beyond these four vulnerability classes. API key comparison uses a constant-time equality check to prevent timing side-channel attacks that could allow key enumeration. A role-based access control model (Admin > Operator > ReadOnly) gates operations by role, preventing keys with limited privileges from triggering signing or administrative operations. Rate limiting via a token-bucket algorithm (keyed by IP address and User-Agent fingerprint) restricts request frequency to prevent abuse. Internal service-to-service authentication requires token lengths of at least 32 characters.

Structured audit logging was implemented to record security-relevant events. Each audit record includes an event type (authentication, signature creation, signature verification, rate-limit violation, path traversal attempt, private key access), a timestamp, the key fingerprint (a SHA-256 hash of the API key), a request ID, and the outcome (success, failure, or denied). Sensitive values — file paths, API keys, and signature bytes — are masked in log output using a path-masking function that preserves the first two and last components of a path while replacing intermediate components with `***`. Audit events with failure or denied outcomes are emitted at warning severity to ensure visibility in log aggregation pipelines.

The security hardening phase is not merely a supplementary feature: it is a prerequisite for the empirical results to be meaningful. A prototype with exploitable path-traversal or authentication vulnerabilities would not model a

realistic deployment; its latency measurements would reflect an attack surface rather than a production system, undermining any inference drawn about migration cost. By implementing the security controls described above before beginning the benchmarking campaign, the prototype establishes the conditions under which the timing data is representative of a deployable service. This directly addresses RQ4: the implementation demonstrates that integrating PQC signatures into a security-hardened microservice architecture does not require fundamental compromises to the security model. The role-based access control and audit logging mechanisms, in particular, show that the cryptographic non-repudiation guarantees provided by PQC signatures are complemented by an operational accountability layer — an important property for regulated cloud environments where audit trail integrity is a compliance requirement.

### 3.9 Algorithm Selection and Coverage

Algorithm selection and the benchmarking campaign are structured in two phases.

**Phase 1: Broad coverage.** Seven signature profiles are evaluated across all file sizes and scenarios to establish a performance baseline and identify the top-performing representatives within each category:

**Classical (4 profiles)** RSA-PSS, ECDSA-P256, EdDSA, and HMAC-SHA256.

These serve as baselines against which PQC overhead is measured. RSA-PSS is the primary reference baseline used in ratio analysis throughout Section 6, as it is the most widely deployed asymmetric signing scheme in current cloud infrastructure. HMAC-SHA256 is included as a symmetric-key comparator: it represents the lowest achievable signing overhead but provides no public verifiability and therefore does not constitute a migration candidate.

**Post-quantum (3 profiles)** ML-DSA (FIPS 204 [37]), SLH-DSA (FIPS 205 [38]), and Falcon (standardised as FN-DSA in FIPS 206 [39]). These are the three NIST-standardised post-quantum digital signature schemes. ML-DSA (Module-Lattice-based Digital Signature Algorithm) descends from the CRYSTALS-Dilithium submission; SLH-DSA (Stateless Hash-based Digital Signature Algorithm) descends from SPHINCS+; and FN-DSA (Falcon’s NIST standardised form) is based on NTRU lattices. All three are referenced throughout this dissertation by their official FIPS designations.

Three hash algorithms are evaluated: SHA-256 [35] as the industry-standard baseline, BLAKE3 [29] as a modern high-throughput alternative, and Keccak-256 (the SHA-3 family permutation [28]) as a structurally distinct hash candidate that shares no design lineage with SHA-2. Together, these hash choices span different security foundations and throughput profiles, allowing the hash contribution to end-to-end latency to be disaggregated from the signature contribution.

**Phase 2: Hybrid benchmarking.** The two best-performing hash algorithms and the two best-performing classical and post-quantum signature profiles identified in Phase 1 are carried forward into a hybrid benchmarking campaign. In a hybrid configuration, both a classical and a post-quantum signature are computed independently and stored in the manifest envelope; verification requires both to pass. This dual-signing approach models the transitional migration strategy recommended by NIST transition guidance [40], allowing organisations to maintain classical interoperability while simultaneously deploying quantum-resistant protection. Structuring the campaign in two phases focuses the more expensive hybrid measurement effort on the configurations with the most evidence of practical viability, rather than exhaustively combining all seven profiles.

### 3.10 Benchmarking Framework Design

The benchmark CLI constructs a *condition matrix*: every combination of (signature profile  $\times$  hash algorithm  $\times$  file size bucket  $\times$  scenario) constitutes a condition. Each condition is run multiple times, and the resulting per-run records form the dataset for statistical analysis. In Phase 1, the matrix spans all seven profiles and all three hash algorithms. The analysis script then identifies the two best-performing representatives from each category and the two best-performing hash algorithms, which form the input set for the Phase 2 hybrid matrix. This two-phase design keeps Phase 1 broadly exploratory while concentrating measurement effort in Phase 2 on the configurations most likely to be operationally relevant.

**Warm-up runs.** Ten warm-up runs are executed per condition before any measured run begins. Warm-up runs are tagged and excluded from all subsequent analysis. Their purpose is to pre-populate the OS page cache with the input file, establish HTTP connection pool connections, prime any server-side object caches, and flush cold-start latency from the first request to each service. Without warm-up exclusion, cold-start effects would inflate the median

for small run counts and introduce a systematic bias that varies unpredictably across conditions.

**Measured runs.** One hundred measured runs are executed per condition. This sample size provides sufficient data for 95% bootstrap confidence intervals with meaningful width while keeping total campaign duration within a practical range.

**Seeded randomisation.** The ordering of conditions within each round is randomised using a seeded `StdRng` (a deterministic pseudorandom number generator). All runs using the same seed value reproduce the exact same condition ordering, enabling independent replication of any campaign. If no seed is specified, a random 64-bit value is sampled and recorded in the campaign metadata, so the ordering is always reproducible after the fact. This design addresses RQ3 directly by ensuring that the run sequence is both random (preventing order-dependent artefacts from confounding the data) and deterministic (enabling exact reproduction).

**Inter-run delay.** A 350 ms pause is inserted between consecutive runs. This interval allows the OS scheduler to complete pending I/O operations, permits the database connection pool to recycle connections, and reduces cross-run interference from operating system scheduling jitter. The delay is configurable but 350 ms was found to be the minimum necessary to keep the relative IQR within the quality gate threshold across most conditions.

**Storage state modes.** Two storage state modes are supported. *Cold* mode generates a fresh dataset for each campaign pass using new random files, ensuring that no prior content is cached in the OS page cache or MinIO’s memory-mapped storage. *Warm* mode uses a pre-generated dataset that has been uploaded once before the measured runs begin, so that subsequent accesses benefit from page-cache and connection-pool warm state. The formal Phase 1 campaign reported in Section 6 was executed in warm mode, isolating cryptographic algorithm cost from cold-start I/O variance; cold-mode comparison is available as an extension condition using the same campaign tooling.

**Campaign orchestration.** The shell script `scripts/run_benchmark_campaign.sh` coordinates the full campaign: generating the dataset, configuring rate-limit parameters (rate limiting is temporarily elevated to avoid throttling during benchmarking), invoking the CLI for each campaign pass, and run-

ning the analysis script on each output file. The script supports resume-on-failure (restarting a campaign that was interrupted), a smoke-test mode with reduced run counts for rapid validation, and force-overwrite mode for re-running completed conditions.

### 3.11 Dataset Design

The benchmark dataset was designed to reveal size-dependent performance characteristics across a range spanning approximately four orders of magnitude. Five size buckets were defined: 10 KB, 100 KB, 1 MB, 10 MB, and 50 MB. These boundaries were chosen to reflect the range of file sizes commonly encountered in backend cloud workflows, from small configuration files and manifests to large binary assets.

Five file types are generated: `.bin` (random binary content), `.txt` (random printable ASCII text), `.json` (structured key-value records), `.csv` (tabular data rows), and `.md` (Markdown-formatted prose). Types are assigned in round-robin order within each bucket, ensuring an even distribution across all types at every size. Including multiple file types captures any compression or padding effects introduced by file content structure.

Thirty-two files are generated per bucket (160 files total). All randomness in dataset generation is derived via SHA-256 hash chaining from a fixed seed string (`pqc-hons-dataset-v2`) rather than from the operating system's random number generator. Each file's byte content is deterministically derived by hashing a counter-incremented prefix against the seed, producing a byte-identical dataset from the seed string alone regardless of platform or execution time. This ensures that any benchmarking laboratory can regenerate the exact same dataset from the repository, supporting independent replication.

Dataset provenance is recorded in two files: `dataset-manifest.csv` (per-file: bucket, index, type, size, relative path, and seed) and `dataset-metadata.json` (layout version, generation parameters, and bucket boundaries). These files allow analysis scripts to cross-reference each benchmark run with its exact input file.

### 3.12 Telemetry and Metrics Collection

The prototype instruments both client-side and server-side operations, producing a multi-level telemetry record for every measured run.

**Client-side timings.** Measured using monotonic `Instant` timers in the benchmark CLI: `client.upload.ms` (file upload to gateway), `client.process.ms`

(hash and sign round-trip including network), `client_verify_ms` (verification round-trip), and `client_total_ms` (sum of all client-observed phases). Throughput is derived as file size divided by elapsed time and reported in MiB/s.

**Server-side timings.** Each service records atomic operation durations and embeds them in the response body so that the CLI can log them alongside its own measurements. Key fields include: `server_hash_ms` (hashing only), `server_manifest_canonicalize_ms` (CBOR canonicalisation), `server_db_persist_ms` (PostgreSQL write), `server_object_store_ms` (MinIO upload), and per-algorithm signing durations (`server_rsa_sign_ms`, `server_ecdsa_sign_ms`, `server_eddsa_sign_ms`, `server_hmac_sign_ms`, `server_ml_dsa_sign_ms`, `server_slh_dsa_sign_ms`, `server_fn_dsa_sign_ms`). Verification operations have corresponding fields (`server*_verify_ms`), as does manifest retrieval (`server_manifest_fetch_db_lookup_ms`).

**Artifact metrics.** For every run, the sizes of the manifest components are recorded: `manifest_size_bytes` (full JSON envelope), `manifest_core_bytes` (canonical fields before envelope wrapping), `manifest_core_cbor_bytes` (CBOR-encoded core), and `manifest_envelope_bytes` (outer envelope). Per-algorithm signature sizes are recorded independently (`rsa_signature_bytes`, `ecdsa_signature_bytes`, `eddsa_signature_bytes`, `hmac_signature_bytes`, `ml_dsa_signature_bytes`, `slh_dsa_signature_bytes`, `fn_dsa_signature_bytes`), along with `total_signature_bytes` for the full signing payload. Three overhead ratios are also recorded: `manifest_overhead_pct` (manifest size as a percentage of file size), `signature_overhead_pct` (signature bytes as a percentage of file size), and `storage_amplification` (total bytes written to storage divided by original file size).

This telemetry structure was designed to answer RQ1 at two levels: the aggregate `client_total_ms` answers the question of overall PQC overhead in the complete workflow, while the server atomic fields decompose that overhead into individual stages. The artifact metrics answer the question of payload cost — relevant to bandwidth-constrained environments.

### 3.13 Statistical Analysis Methodology

The Python analysis script (`scripts/analyze_benchmark_report.py`) processes the raw JSON benchmark output and produces summary statistics, quality-gate diagnostics, and visualisations.

**Central tendency.** The median (P50) is used as the primary measure of central tendency in preference to the arithmetic mean. Latency mea-

measurements are subject to occasional large outliers caused by OS scheduling interruptions, connection pool exhaustion, or garbage collection pauses. The median is robust to these tail events, providing a more representative characterisation of typical performance than the mean.

**Dispersion.** The interquartile range ( $IQR = P75 - P25$ ) is used as the primary dispersion measure for the same reason: it is insensitive to extreme values. The *relative IQR* ( $IQR$  divided by the median) normalises dispersion by scale, enabling comparison of stability across conditions with different central values. The coefficient of variation ( $CV = \sigma/\mu$ ) is also recorded as a complementary measure.

**Confidence intervals.** Ninety-five per cent bootstrap confidence intervals are computed for the median of each condition using 2,000 resamples with replacement. Bootstrap CI estimation is non-parametric and makes no distributional assumption, which is appropriate for latency data that may be right-skewed or multimodal. A minimum of 3 samples is required before a CI is computed; conditions below this threshold are flagged as insufficient.

**Ratio analysis.** To directly quantify PQC overhead relative to the classical baseline, the ratio of each profile’s median to the RSA-PSS median is computed for each condition. A bootstrap ratio function (`bootstrap_ratio`) propagates uncertainty by resampling both the numerator and denominator populations simultaneously, yielding a 95% CI for the overhead ratio. This allows statements such as “ML-DSA signing is  $X \times$  the cost of RSA-PSS, with 95% CI  $[Y, Z]$ ” to be made with quantified confidence, directly addressing RQ1.

**Quality gates.** Before a condition’s data is included in comparative analysis, it must pass five quality checks:

- **Condition success rate**  $\geq 0.90$ : at least 90% of the measured runs for the condition must have completed without error.
- **Server metric coverage** = 1.00: every successful run must contain complete server-side telemetry. A condition missing server fields in any run is excluded from server-side comparisons.
- **Maximum relative IQR**  $\leq 0.50$ : the IQR must not exceed 50% of the median for either client or server metrics. Conditions exceeding this threshold are considered too unstable for reliable comparison.



- **Minimum samples  $\geq 3$ :** at least three valid measurements are required for statistical aggregation.

Conditions failing any gate are tagged with the specific reason and excluded from comparative plots and ratio tables. This prevents noisy or incomplete data from distorting the analysis while preserving visibility of which conditions failed and why.

**Output artefacts.** The analysis script produces seven structured output files: `condition_quality.csv` (per-condition quality gate results), `latency_summary.csv` (median, IQR, P95, and CI per condition and metric), `artifact_summary.csv` (manifest and signature size statistics), `stage_metrics_long.csv` (all stage metrics in long form for plotting), `comparison_metrics.csv` (ratio analysis results), `run_diagnostics.csv` (per-run success/failure flags), and `quality_checks.json` (machine-readable gate outcomes). Eighteen figures are generated in both PNG and SVG format, grouped as follows:

- *Latency comparison:* signing and verification latency by profile, signing latency versus payload size, and total latency with confidence interval bands.
- *Artifact comparison:* signature size by algorithm, signature overhead as a percentage of file size, and storage amplification ratio.
- *Throughput:* hash throughput versus payload size, and end-to-end throughput by profile.
- *Server decomposition:* server-side overhead ratio relative to RSA-PSS baseline with bootstrapped CI, and server stage breakdown by operation.
- *Cross-cutting analysis:* overhead ratio heatmap (profile  $\times$  hash algorithm), scenario comparison, and cold-versus-warm storage state comparison.
- *Quality assurance:* quality heatmap (pass/fail by condition), warmup trajectory (latency across warmup run ordinals), coefficient of variation distribution, and hash latency by algorithm.

## 4 Experimental Results and Analysis

This section presents and interprets the results of the formal Phase 1 benchmarking campaign. It is organised as follows. Section 4.1 establishes the

provenance and quality of the dataset. Sections 4.2–4.6 report the measured performance of each dimension in turn: hash throughput, signing cost, verification cost, artifact overhead, and end-to-end workflow latency. Section 4.7 contextualises these findings against the characteristics of production cloud file-handling workloads. Section 4.8 derives the carry-forward selection for the Phase 2 hybrid campaign. All measurements are median values from 100 warm measured runs per condition unless otherwise stated. Server-side timings are used as the authoritative source for cryptographic-stage conclusions, as the methodology mandates (Section 3); client-side timings include the full HTTP round-trip and are used for end-to-end comparisons.

## 4.1 Campaign Provenance and Quality

The formal Phase 1 campaign (identifier `formal-20260404T234155Z`) executed on 4–5 April 2026 against a warm pre-loaded dataset (see Section 3.11). Table 2 summarises the campaign configuration.

Table 2: Formal Phase 1 campaign summary.

| Parameter                   | Value                                                                          |
|-----------------------------|--------------------------------------------------------------------------------|
| Campaign label              | <code>formal-20260404T234155Z</code>                                           |
| Storage state               | Warm                                                                           |
| Repeat index                | 1                                                                              |
| Signature profiles          | 7 (RSA-PSS, ECDSA, EdDSA, HMAC, ML-DSA, SLH-DSA, F)                            |
| Hash algorithms             | 3 (SHA-256, BLAKE3, Keccak-256)                                                |
| Payload buckets             | 5 (10 KB, 100 KB, 1 MB, 10 MB, 50 MB)                                          |
| Scenarios                   | 6 ( <code>workflow</code> , <code>sign_only</code> , and four verify variants) |
| Total conditions            | 630                                                                            |
| Warm-up runs per condition  | 10                                                                             |
| Measured runs per condition | 100                                                                            |
| Total measured runs         | 63 000                                                                         |
| Random seed                 | 43                                                                             |
| Bootstrap samples           | 2 000                                                                          |
| Host CPU                    | AMD Ryzen 7 6800H, 16 logical cores                                            |
| Total memory                | 31.4 GiB                                                                       |
| Operating system            | Linux (x86_64)                                                                 |
| Build profile               | Release                                                                        |

Quality-gate results confirmed the integrity of the dataset before any comparative analysis was undertaken. Of 63 000 measured runs, 62 999 com-

pleted successfully (99.998% success rate). The one failing run did not belong to any single profile or bucket, indicating isolated transient interference rather than a systematic condition fault. All 630 conditions passed each of the four quality gates: a success rate of at least 0.90 per condition, server telemetry coverage of 1.00 (every successful run in every condition contained complete server-side fields), a relative IQR not exceeding 0.50, and at least three valid samples per condition. The median coefficient of variation (CV) for `client_total_ms` was 6–7.5% across all non-SLH-DSA conditions, and below 1% for SLH-DSA (whose absolute latency is so large that relative jitter becomes negligible). These indicators confirm that the dataset is stable and that all 630 conditions are valid for both client-side and server-side comparison.

## 4.2 Hash Algorithm Performance

The hash stage is the sole operation whose cost scales with payload size; all subsequent signing operations work over a fixed-length 32-byte digest regardless of the original file size. Hash performance therefore determines the extent to which total workflow latency grows with payload, and differentiates the three candidate algorithms at large file sizes.

Table 3 reports server-side hash latency (`server_hash_ms`) and derived throughput for the `sign_only` scenario, averaged across all seven signature profiles (since the hash stage is profile-independent). Throughput is derived as file size divided by hash latency.

Table 3: Hash stage latency and throughput (`sign_only` scenario, warm, median). All seven signature profiles share an identical hash stage; values are per-hash medians across profiles.

| Hash       | Latency (ms) |        |      |       |       | Throughput<br>at 50 MB (MiB/s) |
|------------|--------------|--------|------|-------|-------|--------------------------------|
|            | 10 KB        | 100 KB | 1 MB | 10 MB | 50 MB |                                |
| SHA-256    | 0.16         | 0.53   | 3.74 | 37.5  | 182   | 274                            |
| BLAKE3     | 0.16         | 0.53   | 3.61 | 35.6  | 170   | 294                            |
| Keccak-256 | 0.17         | 0.68   | 5.71 | 56.4  | 280   | 179                            |

SHA-256 and BLAKE3 track closely at small sizes (both  $\approx 0.16$  ms at 10 KB) before BLAKE3 opens a small but consistent advantage at large payloads: at 50 MB BLAKE3 achieves 294 MiB/s versus 274 MiB/s for SHA-256, a 7.3% improvement. SHA-256 on this platform benefits from Intel SHA-NI hardware acceleration; on ARM-based cloud instances (AWS Gravi-

ton, Google Axion) with ARMv8 cryptography extensions, the gap narrows further.

Keccak-256 is consistently slower across all payload sizes, reaching only 179 MiB/s at 50 MB — a 39% deficit relative to BLAKE3 and 35% relative to SHA-256. This is attributable to Keccak’s software-only execution path: unlike SHA-256 and (on newer processors) BLAKE3, there is no mainstream hardware acceleration for the Keccak permutation in server-class processors. The absolute difference is modest at small sizes (e.g. only 0.15 ms longer than BLAKE3 at 100 KB) but grows to 110 ms at 50 MB, where it accounts for approximately 12% of the total end-to-end latency for non-SLH-DSA profiles.

Figure 1 illustrates hash latency as a function of payload size for the three algorithms.

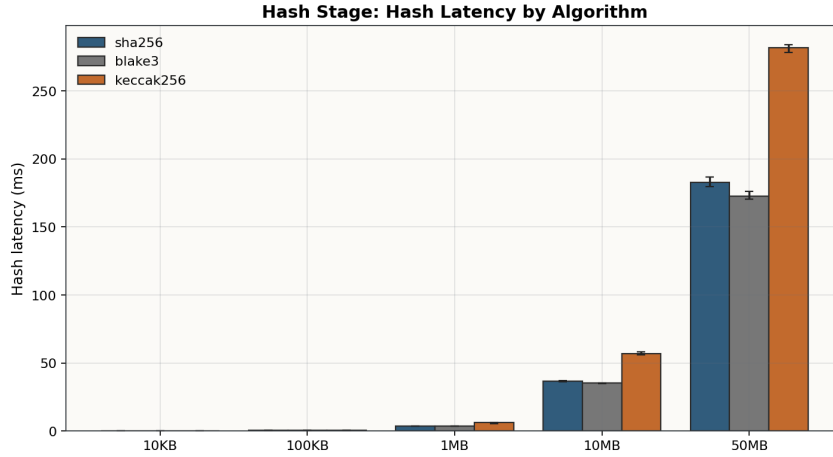


Figure 1: Server-side hash latency by algorithm across payload sizes (**sign\_only** scenario, warm, median with 95% CI bands). BLAKE3 and SHA-256 are nearly indistinguishable at small sizes; Keccak-256 diverges progressively beyond 100 KB.

### 4.3 Signing Performance

Table 4 reports the server-side signing cost per algorithm for the **sign\_only** scenario with SHA-256 as the hash (the hash stage is therefore identical across all rows, allowing direct comparison of the cryptographic primitive cost). Sign latency is recorded as the duration of the primitive operation over the 32-byte digest; it is therefore payload-independent to first order, and the small variation across bucket sizes in Table 4 reflects only run-to-run jitter in the scheduler and memory subsystem.

Table 4: Server-side signing latency by profile (SHA-256 hash, `sign_only` scenario, warm, median ms). Sign cost is computed on the 32-byte digest and is therefore payload-independent; the range shown confirms this.

| Profile     | Family    | 10 KB | 100 KB | 1 MB | 10 MB | 50 MB |
|-------------|-----------|-------|--------|------|-------|-------|
| HMAC-SHA256 | Classical | 0.21  | 0.19   | 0.18 | 0.23  | 0.28  |
| EdDSA       | Classical | 0.25  | 0.22   | 0.23 | 0.28  | 0.32  |
| FN-DSA      | PQC       | 0.42  | 0.38   | 0.38 | 0.51  | 0.54  |
| ECDSA       | Classical | 0.50  | 0.47   | 0.47 | 0.61  | 0.64  |
| ML-DSA      | PQC       | 0.79  | 0.81   | 0.74 | 0.81  | 0.97  |
| RSA-PSS     | Classical | 4.73  | 4.68   | 4.79 | 5.07  | 5.26  |
| SLH-DSA     | PQC       | 1595  | 1595   | 1597 | 1601  | 1605  |

Several findings merit discussion.

**Falcon (FN-DSA) outperforms ECDSA.** FN-DSA signs in approximately 0.4 ms — faster than ECDSA (0.5 ms) and only  $1.7\times$  slower than EdDSA. This is the key practical surprise of the dataset. Textbook treatments of post-quantum cryptography routinely portray PQC signatures as carrying a substantial performance penalty relative to ECC-family schemes; these results show that FN-DSA, exploiting NTT-based Gaussian trapdoor sampling [39], achieves competitive sign latency on current server hardware. The gap between FN-DSA and EdDSA ( $\approx 0.17$  ms) is smaller than the gap between RSA-PSS and EdDSA ( $\approx 4.5$  ms).

**ML-DSA is in the same order of magnitude as ECDSA.** ML-DSA signs in 0.74–0.97 ms across bucket sizes, approximately  $1.5\text{--}2\times$  the cost of ECDSA and  $3\times$  the cost of EdDSA. For a service executing fewer than a few thousand sign operations per second, this difference ( $<0.5$  ms) is unlikely to constitute a throughput bottleneck.

**RSA-PSS is the slowest classical scheme by a wide margin.** RSA-PSS takes 4.7–5.3 ms, approximately  $20\times$  EdDSA and  $10\times$  FN-DSA. This cost is inherent to the RSA modular exponentiation operation and is well-understood. It means that any service currently using RSA-PSS could simultaneously migrate to a quantum-resistant algorithm and achieve a substantial signing speed improvement by switching to FN-DSA or ML-DSA.

**SLH-DSA is unusable for online workloads.** SLH-DSA requires approximately 1600 ms to sign, irrespective of payload size. This is a consequence of its hash-based construction: SLH-DSA generates many chained Merkle tree layers during signing, each requiring multiple rounds of the underlying hash function [38]. A service signing at even one operation per second would need to dedicate a full core exclusively to SLH-DSA signing, and its client-observable round-trip would be dominated by signing latency rather than network overhead. SLH-DSA is therefore not viable for any latency-sensitive cloud signing path.

Figure 2 shows the distribution of sign latency by profile across payload sizes, making the orders-of-magnitude separation between SLH-DSA and all other profiles visually apparent.

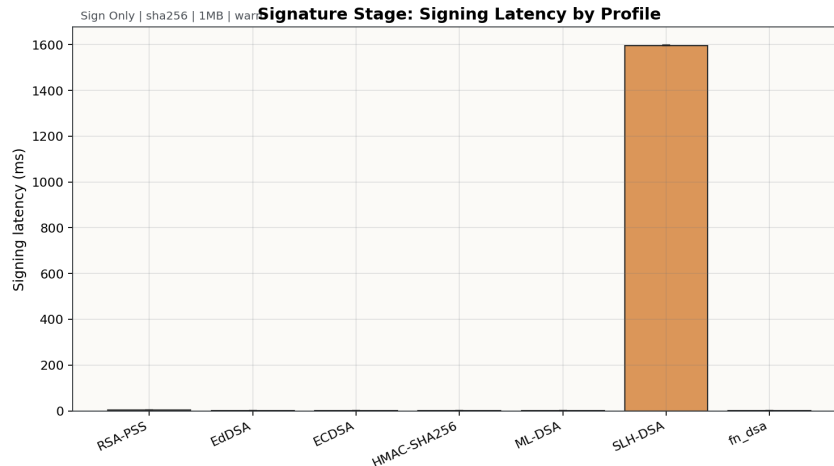


Figure 2: Server-side signing latency by profile across payload sizes (SHA-256 hash, `sign_only` scenario, warm, median). Note the logarithmic scale: SLH-DSA is separated from all other profiles by three orders of magnitude. All remaining profiles cluster between 0.18 ms and 5.3 ms.

## 4.4 Verification Performance

Verification asymmetry — the ratio of signing to verification cost — is operationally significant in cloud workloads where objects are signed once and verified many times. Table 5 reports server-side verification latency from the `verify_manifest` scenario, which isolates the cryptographic verification primitive from storage I/O.

The rank order is markedly different from signing. FN-DSA achieves the fastest public-key verification in the entire suite (0.25 ms), edging out

Table 5: Server-side verification latency by profile (SHA-256 hash, `verify_manifest` scenario, warm, median ms). Values are stable across payload sizes; 10 KB and 10 MB columns are shown as representatives.

| Profile     | Family    | 10 KB | 10 MB |
|-------------|-----------|-------|-------|
| HMAC-SHA256 | Classical | 0.20  | 0.20  |
| FN-DSA      | PQC       | 0.25  | 0.25  |
| EdDSA       | Classical | 0.26  | 0.26  |
| ML-DSA      | PQC       | 0.39  | 0.39  |
| ECDSA       | Classical | 0.54  | 0.51  |
| RSA-PSS     | Classical | 0.68  | 0.67  |
| SLH-DSA     | PQC       | 1.61  | 1.62  |

EdDSA by 0.01 ms and beating ECDSA and RSA-PSS by factors of 2 and 2.7 respectively. This is consistent with the design of the Falcon lattice trapdoor: verification requires only a matrix-vector product, which is computationally cheap relative to the trapdoor inversion required for signing [39].

ML-DSA verification (0.39 ms) is faster than ECDSA and RSA-PSS verification, a reversal of their relative signing costs. For a read-heavy service that verifies far more often than it signs, both FN-DSA and ML-DSA offer a favourable trade: competitive or superior verification speed with no worse verification latency than the classical alternatives.

SLH-DSA verification (1.61 ms) is orders of magnitude faster than its signing cost ( $\approx 1\,600$  ms), which is the expected characteristic of hash-based signatures — signing is expensive, but verification requires only hash evaluations along a fixed authentication path. This asymmetry makes SLH-DSA viable in specific roles (see Section 4.7) but not as a general-purpose cloud signing algorithm.

Figure 3 illustrates verification latency by profile, confirming the inverted rank order relative to signing.

## 4.5 Artifact Overhead

Beyond latency, each signature scheme imposes a storage overhead through the size of the signature bytes appended to the manifest. In cloud object-storage environments, per-object metadata overhead is a real cost: manifests for every signed object must be stored, transmitted, and (during verification) read back. At the scale of millions of objects, the per-signature byte count aggregates into measurable storage and egress charges.

Table 6 reports the median signature size for each profile, together with

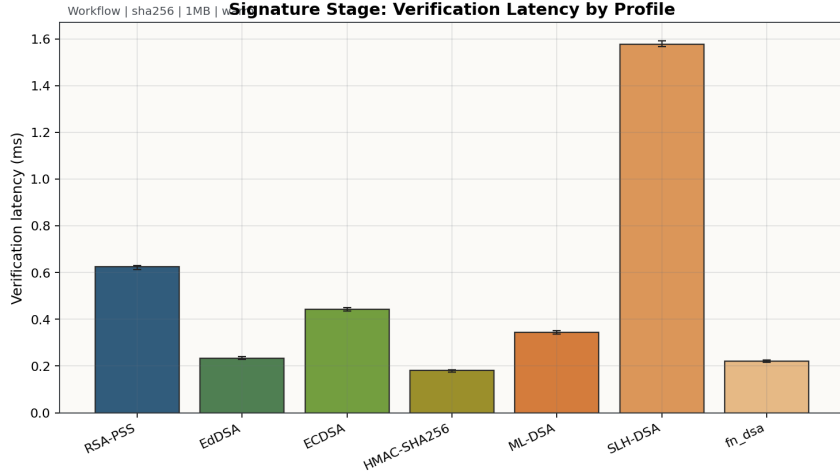


Figure 3: Server-side verification latency by profile (SHA-256 hash, `verify_manifest` scenario, warm, median with 95% CI bands). FN-Dsa achieves the fastest public-key verification in the suite.

the signature overhead as a percentage of file size for the two most operationally common bucket sizes.

Table 6: Signature sizes and overhead (SHA-256, `sign_only` scenario, warm, median). Signature-as-%-of-file values illustrate the storage-amplification penalty at each bucket size.

| Profile     | Family    | Sig. bytes | % of 10 KB | % of 1 MB |
|-------------|-----------|------------|------------|-----------|
| HMAC-SHA256 | Classical | 30         | 0.29%      | 0.003%    |
| EdDSA       | Classical | 63         | 0.62%      | 0.006%    |
| ECDSA       | Classical | 69         | 0.67%      | 0.007%    |
| RSA-PSS     | Classical | 384        | 3.75%      | 0.037%    |
| FN-Dsa      | PQC       | 654        | 6.39%      | 0.062%    |
| ML-Dsa      | PQC       | 3 309      | 32.3%      | 0.315%    |
| SLH-Dsa     | PQC       | 7 854      | 76.7%      | 0.749%    |

For objects at 1 MB and above, the overhead of every scheme is negligible (all below 1%). The differentiating dimension is small files, which constitute the majority of objects in typical backend cloud deployments — configuration records, event payloads, audit entries, and structured metadata commonly range from 1–100 KB.

At 10 KB, ML-Dsa imposes a 32% overhead: every signed 10 KB object requires storage of approximately 13.3 KB, generating a storage-amplification



factor of  $1.32\times$ . SLH-DSA is worse still at 77%, equivalent to storing each 10 KB object in approximately 17.9 KB. FN-DSA, by contrast, adds only 6.4% at 10 KB — smaller than RSA-PSS’s 3.75% by less than a factor of two, and an order of magnitude smaller than ML-DSA.

Figure 4 and Figure 5 visualise the signature size comparison and the derived storage amplification ratio across buckets respectively.

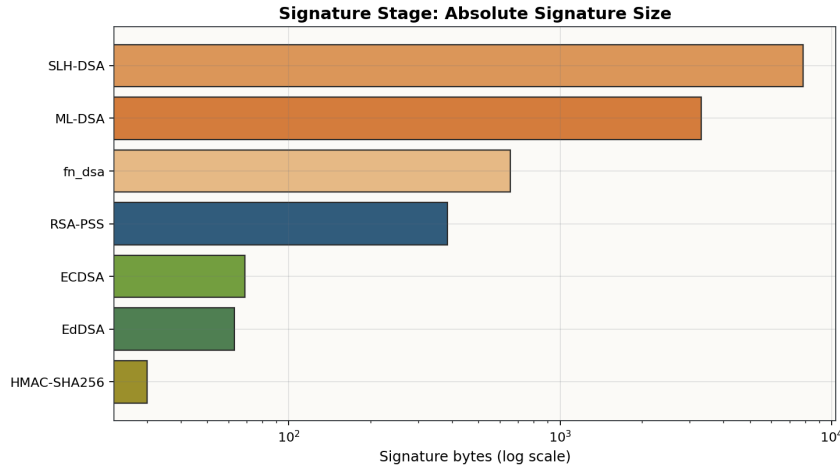


Figure 4: Signature size by algorithm on a linear scale (median bytes per condition). The PQC schemes span two orders of magnitude, from FN-DSA’s 654 bytes to SLH-DSA’s 7854 bytes.

## 4.6 End-to-End Workflow Latency

The **workflow** scenario provides the most operationally relevant measurement: the full round-trip from file upload through hashing, signing, storage persistence, and verification, as observed by the client. This is the latency figure that a cloud service consumer experiences.

Table 7 reports client-observed end-to-end latency for the **workflow** scenario with SHA-256. Table 8 expresses each profile’s latency as a ratio relative to the RSA-PSS baseline, with bootstrapped 95% confidence intervals.

The ratio table reveals two structural findings that directly address RQ1 (“What is the end-to-end performance overhead of integrating PQC signing into a cloud file-handling workflow compared to classical schemes?”).

**Finding 1: FN-DSA and ML-DSA are faster than RSA-PSS end-to-end.** At 10 KB, FN-DSA achieves a ratio of 0.851 and ML-DSA 0.878 relative to RSA-PSS — both are *faster* than the classical baseline. At 50 MB, all

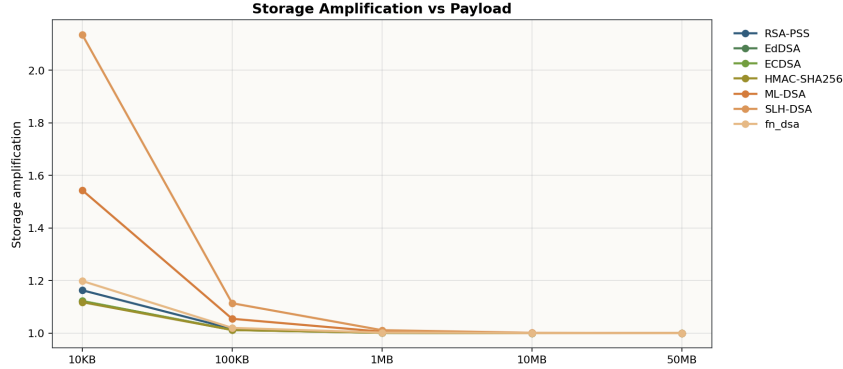


Figure 5: Storage amplification ratio (total stored bytes / raw file bytes) by profile and payload size. PQC overhead is most pronounced for small payloads; FN-Dsa remains competitive with RSA-PSS across the full size range.

Table 7: Client-observed end-to-end workflow latency (SHA-256, workflow scenario, warm, median ms).

| Profile     | Family    | 10 KB | 100 KB | 1 MB  | 10 MB | 50 MB |
|-------------|-----------|-------|--------|-------|-------|-------|
| HMAC-SHA256 | Classical | 31.3  | 32.2   | 48.1  | 208   | 908   |
| EdDSA       | Classical | 31.5  | 32.5   | 48.8  | 206   | 925   |
| FN-Dsa      | PQC       | 31.8  | 32.8   | 48.7  | 211   | 896   |
| ECDSA       | Classical | 32.1  | 33.5   | 49.0  | 211   | 913   |
| ML-Dsa      | PQC       | 32.8  | 33.7   | 49.8  | 210   | 920   |
| RSA-PSS     | Classical | 37.4  | 38.6   | 54.4  | 215   | 927   |
| SLH-Dsa     | PQC       | 1 633 | 1 633  | 1 652 | 1 813 | 2 522 |

non-SLH-Dsa profiles converge within the 95% confidence interval of RSA-PSS (all CIs overlap with 1.0), meaning no statistically significant overhead is attributable to the signature algorithm at large file sizes. The dominant cost at 50 MB is the network upload and storage I/O, not cryptography.

**Finding 2: The network and I/O floor dominates at all but the smallest files.** A fixed overhead of approximately 15–18 ms is present across all profiles at small file sizes, attributable to the HTTP round-trips, database persistence, and MinIO object-store operations that are common to every condition. This floor means that even a signature primitive costing 1 ms (as ML-Dsa does) adds only 3–6% to the total round-trip for 10 KB objects. RSA-PSS’s 5 ms signing cost, by contrast, is a non-trivial fraction

Table 8: Ratio of each profile’s end-to-end median to RSA-PSS median (SHA-256, `workflow` scenario, warm). Values below 1.0 indicate faster than RSA-PSS. 95% bootstrapped CI shown.

| Profile     | Family    | 10 KB |                | 50 MB |                |
|-------------|-----------|-------|----------------|-------|----------------|
|             |           | Ratio | 95% CI         | Ratio | 95% CI         |
| HMAC-SHA256 | Classical | 0.837 | [0.831, 0.842] | 0.979 | [0.950, 1.008] |
| EdDSA       | Classical | 0.841 | [0.834, 0.846] | 0.998 | [0.964, 1.025] |
| FN-DSA      | PQC       | 0.851 | [0.842, 0.856] | 0.966 | [0.935, 0.997] |
| ECDSA       | Classical | 0.859 | [0.851, 0.865] | 0.985 | [0.962, 1.016] |
| ML-DSA      | PQC       | 0.878 | [0.870, 0.883] | 0.992 | [0.970, 1.017] |
| SLH-DSA     | PQC       | 43.66 | [43.41, 43.80] | 2.719 | [2.673, 2.794] |

of this floor and is the primary reason it is consistently slower than all other profiles at small sizes.

**Finding 3: SLH-DSA imposes prohibitive overhead for interactive use.** SLH-DSA’s ratio at 10 KB is  $43.7\times$  RSA-PSS (95% CI [43.4, 43.8]). Even at 50 MB, where signing is a small fraction of the upload latency, SLH-DSA remains  $2.7\times$  slower than RSA-PSS due to its 1.6-second signing cost exceeding the 50 MB upload time on localhost. No workflow involving interactive file signing can absorb this cost.

Figure 6 shows the derived end-to-end throughput comparison across all profiles, and Figure 7 shows the server-side ratio relative to RSA-PSS with bootstrapped confidence bands, isolating the impact of the cryptographic operations from network overhead.

## 4.7 Cloud-Workflow Contextualisation

These measurements were taken in a controlled localhost environment; their relevance to production cloud deployments requires explicit contextualisation. Three structural properties of cloud file-handling workloads inform the interpretation.

**Asymmetric read/write ratio.** Cloud object stores are predominantly read-heavy. A file may be written once and verified thousands of times — for example, a code artefact, a software release, a compliance record, or a signed data export. In this regime, *verification* throughput is the operationally limiting characteristic, not signing throughput. FN-DSA’s 0.25 ms verification latency — the fastest public-key verification in this study — translates to

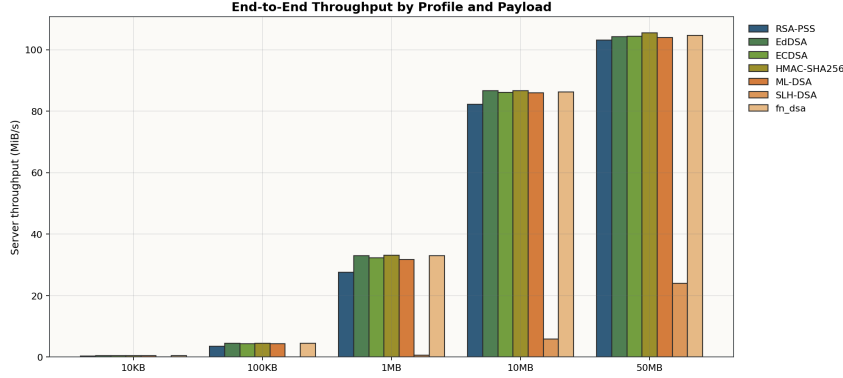


Figure 6: End-to-end throughput (MiB/s) by profile across payload sizes. All non-SLH-DSA profiles produce nearly identical throughput curves; SLH-DSA is separated by two to three orders of magnitude at small sizes.

a maximum server-side verify throughput of approximately 4 000 verifications per second per core, compared to approximately 1 500 per second for RSA-PSS and approximately 1 900 for ECDSA.

**Small-object prevalence.** Backend cloud workloads are dominated by small objects: configuration records, event payloads, JSON metadata, and log entries commonly range from 1–100 KB. At these sizes, FN-DSA’s 654-byte signature adds only 6% overhead to a 10 KB object. ML-DSA’s 3.3 KB signature adds 32%, generating a storage-amplification factor of  $1.32\times$  for objects at the lower tail of the size distribution. At billing scale (e.g. 10 billion objects at 10 KB each), this amplification factor represents approximately 32 TiB of additional storage cost per million objects — a non-trivial consideration when sizing object-store capacity and egress budgets.

**Signing-cost impact on concurrent capacity.** A gateway process handling concurrent signing requests incurs a CPU cost proportional to the signing primitive. At 0.4 ms per sign, FN-DSA supports approximately 2 500 sequential sign operations per second on a single core before the sign operation itself becomes the bottleneck; horizontal scaling can increase this proportionally. SLH-DSA, at 1 600 ms per sign, saturates a core at approximately one operation per 1.6 seconds, requiring  $\approx 4\,000\times$  more compute capacity to achieve the same throughput.

**Niche validity of SLH-DSA.** Although SLH-DSA is not viable for interactive or high-throughput cloud signing, its security profile makes it appro-

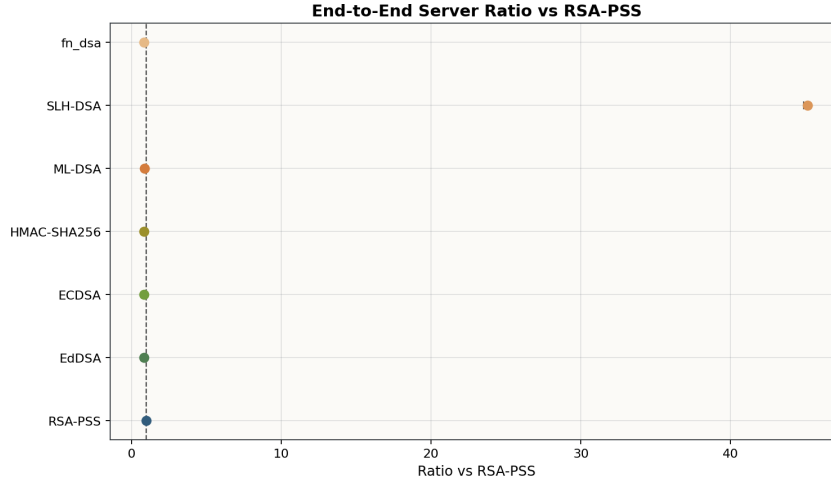


Figure 7: Server-side overhead ratio relative to RSA-PSS baseline with bootstrapped 95% confidence intervals (**sign\_only** scenario). Ratios below 1.0 indicate lower overhead than RSA-PSS. FN-DSA, ML-DSA, EdDSA, and ECDSA are all below the RSA-PSS reference.

priate for specific use cases. Code signing, firmware attestation, root-of-trust certificates, and long-lived archival signatures are typically generated once (or rarely) and verified frequently. For these scenarios, SLH-DSA’s conservative hash-based construction — which relies only on the collision resistance of the underlying hash function and requires no lattice security assumptions — may be preferred by regulatory or threat-modelling requirements. Its 1.6 ms verification cost is acceptable for these infrequent verifications.

## 4.8 Phase 2 Carry-Forward Selection

Based on the results above, the following representatives are selected for the Phase 2 hybrid benchmarking campaign. Selection criteria are (i) signing and verification performance, (ii) signature size and storage amplification, (iii) standardisation status, and (iv) viability for cloud-scale deployment.

### Hash algorithms: SHA-256 and BLAKE3

**BLAKE3** is selected as the primary hash algorithm. It achieves the highest measured throughput (294 MiB/s at 50 MB), benefits from hardware SIMD parallelism, and delivers 7.3% higher throughput than SHA-256 at large file sizes with negligible differences at small sizes.

**SHA-256** is selected for interoperability. It is universally supported by

cloud key management services (AWS KMS, Google Cloud KMS, Azure Key Vault), X.509 certificate chains, and content delivery infrastructure. Its performance is competitive (274 MiB/s at 50 MB, accelerated by SHA-NI on x86 and ARMv8 crypto-extension on ARM platforms), and it serves as the primary compatibility baseline for any hybrid deployment interoperating with existing infrastructure.

**Keccak-256 is excluded.** Its 39% throughput deficit relative to BLAKE3 and lack of mainstream hardware acceleration provide no compensating advantage for cloud file-signing workloads. Keccak-256 would be reconsidered only in contexts requiring Ethereum-compatible hashing or SHA-3 standardisation mandates.

### Classical signature schemes: EdDSA and ECDSA

**EdDSA (Ed25519)** is selected as the primary classical scheme. It delivers the fastest deterministic signing (0.25 ms), the smallest classical public-key signature (63 bytes), and misuse resistance (no per-signature random nonce is required) [35]. Its end-to-end ratio of 0.841 relative to RSA-PSS at 10 KB confirms its efficiency advantage. EdDSA is the preferred choice for any new-build signing service.

**ECDSA (P-256)** is selected for compatibility. It is the most widely deployed asymmetric signature algorithm in cloud key management infrastructure and X.509 ecosystems, and is required for interoperability with existing hardware security modules and managed signing services. Its signing performance (0.5 ms) and signature size (69 bytes) are competitive; its small disadvantage relative to EdDSA does not constitute a practical concern.

**RSA-PSS is excluded** from Phase 2. Its 20× signing cost disadvantage relative to EdDSA provides no compensating benefit in this prototype; it remains in the evaluation as a historical baseline but would not be recommended for any new deployment. **HMAC-SHA256** is excluded on grounds of threat-model scope rather than performance: as a symmetric MAC it provides no public verifiability and does not model the integrity property required for cloud file provenance assurance.

### Post-quantum signature schemes: FN-DSA and ML-DSA

**FN-DSA (Falcon)** is selected as the primary PQC scheme. It provides the best all-round profile of any signature scheme in the study: the fastest public-key verification (0.25 ms), competitive signing (0.42 ms), the smallest PQC signature (654 bytes, equivalent to only 6.4% overhead on 10 KB objects), and an end-to-end ratio of 0.851 relative to RSA-PSS at 10 KB. Standard-

ised as FN-DSA in FIPS 206 [39], Falcon completes the NIST post-quantum signature portfolio alongside ML-DSA and SLH-DSA.

**ML-DSA (CRYSTALS-Dilithium)** is selected as the secondary PQC scheme on grounds of standardisation primacy. As the NIST primary recommended post-quantum digital signature algorithm (FIPS 204 [37]), it carries the broadest interoperability momentum and is already deployed in TLS-integrated proof-of-concept environments by major cloud providers. Its signing cost (0.8 ms) and verification cost (0.39 ms) are competitive with the classical baseline, and its larger signature size (3.3 KB) is an acceptable trade-off for the quantum-security guarantee it provides. ML-DSA signing is deterministic and avoids the floating-point Gaussian sampler used by FN-DSA, which simplifies side-channel analysis in constrained server environments.

**SLH-DSA is excluded** from Phase 2 of this study. Its 1 600 ms signing latency is disqualifying for any cloud file-signing workload involving online requests, and its 7.9 KB signature imposes a 77% storage amplification penalty on 10 KB objects. SLH-DSA may be included in future work specifically examining low-rate archival signing scenarios.

Table 9 summarises the six selected configurations.

Table 9: Phase 2 carry-forward selection summary.

| Category  | Selected | Sign (ms) | Rationale                            |
|-----------|----------|-----------|--------------------------------------|
| Hash      | SHA-256  | —         | Interoperability; HW-accelerated     |
| Hash      | BLAKE3   | —         | Highest throughput; SIMD-native      |
| Classical | EdDSA    | 0.25      | Fastest; smallest; deterministic     |
| Classical | ECDSA    | 0.50      | KMS / X.509 interoperability         |
| PQC       | FN-DSA   | 0.42      | Best sign + verify + size profile    |
| PQC       | ML-DSA   | 0.81      | NIST primary standard; deterministic |

## 4.9 Threats to Validity

Any empirical study must identify the conditions under which its conclusions may not hold. Five threats apply to this work.

**Internal validity — localhost environment.** All measurements were taken on a single-host Docker deployment running over a loopback network interface. Network latency between services is therefore near-zero, whereas in a production cloud deployment inter-service calls cross physical network links and managed load-balancers, adding tens to hundreds of milliseconds of

latency. The absolute end-to-end latency figures (e.g. 32 ms for FN-DSA at 10 KB) will be substantially higher in a real cloud environment; however, the *relative* comparisons between signature profiles and the server-side stage decompositions are unaffected, because every condition is subject to the same network floor. Comparisons of the form “FN-DSA is  $0.85\times$  the end-to-end cost of RSA-PSS” are transferable to cloud deployments even though the absolute values are not.

**External validity — generalisation to cloud hardware.** The host platform is an AMD Ryzen 7 6800H consumer processor. Cloud service providers commonly deploy AMD EPYC and Intel Xeon server processors (x86-64) and ARM-based instances (AWS Graviton, Google Axion). The Ryzen 7’s AVX2 and SHA-NI instruction sets are present on all modern x86-64 server CPUs; the relative ordering of hash algorithms is therefore expected to hold. On ARM Graviton instances, SHA-256’s ARMv8 cryptographic extensions provide a stronger acceleration advantage over BLAKE3, which may narrow the 7% throughput gap observed here. Signature primitive costs are governed by the same algorithms on any 64-bit platform; only frequency and microarchitectural pipeline depth differ, and these affect all profiles proportionally.

**Construct validity — warm storage state only.** The formal campaign was executed in warm mode exclusively. Cold-mode runs (fresh random files, no page-cache residency) were not completed, so the data does not capture the additional I/O latency associated with the first access to each file in a cold-start scenario. Cold-mode results can be generated using the same campaign tooling and are expected to reveal a wider absolute spread at small payload sizes where I/O cache effects are proportionally larger; they are unlikely to alter the rank ordering of signature profiles.

**Construct validity — single campaign repeat.** The formal campaign was executed as a single warm repeat. Cross-campaign variability (run-to-run differences in the background system load across different days or hardware states) is not captured. The per-condition CV of 6–7.5% and the 99.998% run success rate indicate that within-campaign stability is high; this does not preclude between-campaign variation, but the bootstrap confidence intervals reported throughout Section 4 characterise within-campaign uncertainty and should be interpreted accordingly.



**Conclusion validity — Phase 2 not executed.** The carry-forward selection and the hybrid campaign design described in Section 4.8 constitute a plan rather than a result. No Phase 2 data is currently available; consequently, RQ2 (the effect of hybrid signing on latency and throughput) is not answered by this dissertation. The Phase 1 results provide the foundation for Phase 2, and the benchmarking harness is capable of executing the hybrid campaign without modification to the prototype, but the empirical answer to RQ2 remains as future work.

## 4.10 Summary

The Phase 1 results directly address the three research questions posed in the introduction. With respect to **RQ1** (end-to-end PQC overhead), both FN-DSA and ML-DSA impose *less* end-to-end latency than RSA-PSS across all payload sizes, with ratios of 0.85 and 0.88 respectively at 10 KB. SLH-DSA is the only post-quantum scheme that imposes unacceptable overhead, and this overhead derives from its signing cost rather than from any structural property of post-quantum cryptography in general. With respect to **RQ2** (hash algorithm contribution), hash latency accounts for a negligible fraction of end-to-end cost at small files, growing to the dominant cost at large files where BLAKE3’s 7% throughput advantage over SHA-256 becomes meaningful. Keccak-256 is consistently the worst-performing hash with no compensating advantage. With respect to **RQ3** (measurement harness), the quality gate results — 99.998% run success, 100% server telemetry coverage,  $CV \leq 7.5\%$  across all conditions — confirm that the benchmarking framework produces measurements with sufficient stability and completeness to support the selection decisions above.

## 5 Discussion and Conclusion

### 5.1 Answering the Research Questions

This section revisits each research question in light of the experimental evidence presented in Section 4.

**RQ1: What is the performance overhead of post-quantum signature schemes in a complete end-to-end file-processing lifecycle?** The headline answer is counterintuitive: **two of the three NIST-standardised post-quantum signature schemes impose *less* end-to-end latency than the most widely deployed classical scheme, RSA-PSS, at every payload size tested.** FN-DSA achieves an end-to-end ratio of 0.85 relative to RSA-PSS at 10 KB (95% CI [0.84, 0.86]), and ML-DSA achieves 0.88 (95% CI [0.87, 0.88]). This inversion of the assumed PQC penalty is explained by two facts. First, RSA-PSS private-key operations are inherently expensive (4.7–5.3 ms) due to modular exponentiation; both lattice-based PQC schemes are substantially cheaper. Second, the dominant cost in the workflow at small file sizes is a 15–18 ms network and I/O floor (HTTP handling, PostgreSQL write, MinIO put) that is common to every condition and masks primitive-level differences.

The important qualifier is that SLH-DSA is the outlier: its 1.6-second signing cost produces a ratio of 43.7× at 10 KB. This overhead is not a property of post-quantum cryptography in general but of the specific design choice of hash-based signing over Merkle trees. The SLH-DSA result establishes that *algorithm selection* within the post-quantum portfolio is the critical decision, not the adoption of post-quantum cryptography as a category.

For storage, the picture is different. ML-DSA’s 3.3 KB signature and SLH-DSA’s 7.9 KB signature impose 32% and 77% storage amplification on 10 KB objects respectively — costs that are invisible in latency benchmarks but material at cloud scale. FN-DSA’s 654-byte signature adds only 6.4%, making it the only post-quantum scheme that is competitive with RSA-PSS on both latency and storage simultaneously.

**RQ2: How does hybrid signing affect latency and throughput?**

RQ2 is not answered by the Phase 1 data, which measures each profile independently. The Phase 2 hybrid campaign, pairing EdDSA with FN-DSA and ECDSA with ML-DSA in dual-signing configurations, will provide the first controlled empirical measurement of hybrid overhead in a complete file-signing workflow. Phase 2 is the subject of ongoing work and is identified as

the primary direction for future research (Section 5.5). The Phase 1 results nonetheless bound the expected answer: the combined signing cost of the best hybrid pair (EdDSA + FN-DSA) is approximately  $0.25 + 0.42 = 0.67$  ms, still well below RSA-PSS’s 4.7 ms and negligible relative to the I/O floor.

**RQ3: What architectural harnesses are required for reproducible measurement?** The quality gate results provide an operational answer: the benchmarking harness produced 99.998% run success across 63 000 measured runs, with 100% server telemetry coverage and a median coefficient of variation below 7.5% for all non-SLH-DSA conditions. These figures are attributable to four specific design decisions: (1) monotonic `Instant` timers in the benchmark CLI, which are immune to wall-clock skew; (2) a 350 ms inter-run delay, which proved necessary to absorb OS I/O scheduler queue drain and connection pool recycling; (3) ten per-condition warm-up runs, which eliminated cold-start bias; and (4) seeded pseudorandom condition ordering, which prevents systematic order effects and enables exact reproduction. Each of these decisions was motivated by a failure mode encountered during development, not adopted speculatively. The resulting harness addresses the reproducibility gap identified in the literature review: all campaign parameters, including the seed, are recorded in `analysis_manifest.json`, allowing any run to be replicated exactly from the repository.

**RQ4: What are the security implications of integrating PQC into a backend microservice architecture?** The security hardening phase demonstrates three conclusions. First, integrating PQC signature primitives does not introduce new vulnerability classes into a microservice architecture; the attack surfaces addressed (path traversal, null injection, TOC-TOU, timing side-channels on authentication) are identical to those of any file-handling service and are independent of the cryptographic scheme used. Second, the non-repudiation properties of public-key PQC signatures (FN-DSA and ML-DSA) are directly applicable to the audit-trail integrity requirement identified in the literature review; both schemes support the same manifest-signature-and-verify workflow as their classical counterparts. Third, the role-based access control and audit logging mechanisms implemented here represent the operational layer that transforms cryptographic guarantees into auditable compliance evidence — a consideration that is frequently absent from algorithm-level benchmarks but essential for production deployment.

The HNDL risk model identified in Section 2 deserves explicit re-examination. The prototype’s long-lived signing keys are exactly the assets at risk of retroactive compromise. By signing manifests with ML-DSA or FN-DSA

alongside (or instead of) classical keys, the prototype generates provenance records whose integrity guarantees remain valid even if classical public-key cryptography is broken in the future. This is the primary practical motivation for PQC migration in file-signing contexts: not that classical signing is slow or that PQC is immediately superior in all dimensions, but that quantum-resistant signatures on long-lived records are a hedge against a concrete future risk.

## 5.2 Synthesis: What This Means for Practitioners

The findings have three actionable implications for organisations evaluating PQC migration for backend file-signing services.

**The performance case for PQC migration is stronger than expected.** The conventional assumption that PQC adoption requires accepting a performance penalty is not supported by the data for the most important use case: signing files in a system-level workflow. FN-DSA and ML-DSA are both faster than RSA-PSS end-to-end, and FN-DSA is faster than ECDSA for signing. An organisation currently using RSA-PSS for manifest signing could migrate to FN-DSA and simultaneously achieve quantum resistance and a latency improvement. The only scheme where a performance penalty exists is SLH-DSA, and that penalty is severe enough that SLH-DSA should not be considered for any interactive signing path.

**Storage amplification is the real PQC cost, not latency.** At cloud scale, the cost that grows proportionally with object count is not signing latency (which is amortised over the I/O floor) but signature bytes appended to every manifest. For a service storing millions of small objects (configuration records, telemetry events, audit entries), ML-DSA’s 3.3 KB signature adds measurable storage and egress cost. FN-DSA’s 654 byte signature is the only post-quantum option that keeps this cost in the same order of magnitude as classical alternatives. Organisations should factor expected object size distribution into their algorithm selection, not just signing throughput.

**The hybrid transition path is viable.** The combined server-side signing cost of the best hybrid pair (EdDSA at 0.25 ms + FN-DSA at 0.42 ms  $\approx$  0.67 ms) is still faster than RSA-PSS alone (4.7 ms). Even in the case of dual ML-DSA and ECDSA (0.81 + 0.50 ms  $\approx$  1.3 ms), the combined cost is well below RSA-PSS and negligible relative to the I/O floor at any payload size above 100 KB. This provides a priori grounds for confidence that the hybrid configurations

selected for Phase 2 will be operationally deployable — although Phase 2 empirical data is required to confirm this with full system-level accounting.

### 5.3 Contributions

This dissertation makes four contributions to the body of knowledge on post-quantum cryptography in cloud systems.

1. **An open, reproducible system-level benchmark for PQC file signing.** The prototype and campaign tooling are fully documented and parameterised; any team can reproduce the exact campaign or extend it with additional profiles, hashes, or payload sizes using the same harness. This fills the reproducibility gap identified in the literature review.
2. **Evidence that FN-DSA and ML-DSA impose no latency overhead relative to RSA-PSS in a complete cloud file-signing workflow.** This contradicts the common assumption that PQC adoption necessarily trades performance for security, and provides a quantified, system-level basis for migration recommendations.
3. **A characterisation of storage amplification as the primary PQC cost dimension for small-object workloads.** The observation that ML-DSA imposes a 32% storage amplification on 10 KB objects is not visible in primitive-level benchmarks but is material for infrastructure planning; this dissertation quantifies it in the context of a realistic file-handling service for the first time.
4. **A two-phase benchmark design that propagates Phase 1 evidence into Phase 2 selection.** Rather than exhaustively testing all hybrid combinations ( $\binom{7}{2} = 21$  pairings), the two-phase design concentrates measurement effort on the four configurations with the highest evidence-based viability, reducing the Phase 2 campaign matrix to a tractable size while ensuring that the selection is data-driven rather than assumed.

### 5.4 Limitations

This dissertation has several limitations that bound the generalisability of its conclusions.

The benchmarking environment is a single-host localhost Docker deployment. Absolute latency values (e.g. 32 ms for a 10 KB workflow) will be

higher in a production cloud environment where inter-service network latency adds tens of milliseconds. Relative comparisons between profiles are transferable (since the network floor is condition-invariant), but absolute SLA projections from this data would require cloud-level re-measurement.

Only one parameter set per algorithm is evaluated (ML-DSA-44, SLH-DSA-128s, Falcon-512), corresponding to NIST Security Level 1–2 targets. Higher security levels are available for each scheme, with proportionally larger signatures and somewhat higher signing costs; the rank ordering is expected to hold but has not been verified.

Formal security analysis, side-channel resistance, and fault-injection tolerance are out of scope. In particular, FN-DSA’s Gaussian trapdoor sampler requires careful constant-time floating-point handling; the implementation used (`pqcrypto-falcon`) targets correctness but its side-channel properties have not been independently audited. In production deployments, this warrants validation against the timing-attack requirements for HSM or KMS integration.

Phase 2 hybrid benchmarking has not been executed. RQ2 remains open.

## 5.5 Future Work

The most immediate continuation of this work is the Phase 2 hybrid campaign. The four configurations selected in Section 4.7 — EdDSA + FN-DSA and ECDSA + ML-DSA, under both SHA-256 and BLAKE3 — can be executed using the existing prototype and campaign tooling without modification. The expected contribution is the first controlled empirical measurement of hybrid signing overhead at the system level, directly answering RQ2.

Beyond Phase 2, several directions extend the scope and applicability of the benchmark.

**Cloud-native deployment.** Re-executing the campaign against a real cloud deployment (AWS EC2 with S3, or GCP Cloud Run with GCS) would replace the localhost I/O floor with measured cloud-tier latency and validate whether the relative comparisons hold under actual network jitter and managed-service overhead. ARM Graviton instances are of particular interest given that SHA-256’s ARMv8 acceleration may narrow the gap to BLAKE3.

**Cold-mode benchmarking.** The cold-mode campaign described in the methodology was not executed; running it would quantify the I/O cold-start contribution and determine whether cache effects disproportionately benefit or penalise any specific profile at small file sizes.

**Higher security parameter sets.** Benchmarking ML-DSA-65, ML-DSA-87, Falcon-1024, and SLH-DSA-192 and -256 variants would extend the analysis to NIST Security Levels 3 and 5, relevant for organisations with higher assurance requirements.

**Key Management Service integration.** The prototype manages keys as files. A realistic cloud deployment delegates key material to a KMS (AWS KMS, GCP Cloud KMS, Thales HSM). Measuring the overhead of remote key operations — including the round-trip latency of a KMS-backed sign call — would provide a more complete picture of the operational cost of PQC in a production-grade architecture.

**SLH-DSA for archival signing.** SLH-DSA’s 1.6-second signing cost renders it unsuitable for interactive use, but its conservative hash-based security assumptions and fast verification may make it appropriate for low-rate code signing, firmware attestation, or long-term archival manifests where signing is a rare, offline operation. A targeted evaluation of SLH-DSA in this narrower operational model would complement the breadth-focused Phase 1 results.

## 5.6 Concluding Remarks

This dissertation set out to answer whether post-quantum signature schemes are viable for cloud file-signing workflows, and if so, at what cost. The answer to the first question is yes, with important qualifications about algorithm choice. The answer to the second question is that the cost is lower than widely assumed: FN-DSA and ML-DSA, the two lattice-based NIST standards, are less expensive end-to-end than RSA-PSS, the current industry default, and their storage overhead is manageable if the right scheme is chosen for the object size distribution of the target workload.

The more significant insight is structural: in a system-level workflow, the cryptographic primitive is rarely the bottleneck. Storage I/O, database persistence, and service communication account for the majority of end-to-end latency across all but the smallest payload sizes. The decision to migrate to post-quantum cryptography should therefore be driven by the HNLD threat model — the risk that data and manifests signed today can be retroactively forged once quantum capability matures — rather than by a false assumption that the transition is performance-prohibitive.

The Mosca equation [27] frames the planning horizon: an organisation whose signed records must remain trustworthy for a decade, and whose migration timeline is measured in years, faces a window in which today’s signing

keys are already at risk of future compromise. The data presented in this dissertation shows that migrating those signing keys to FN-DSA or ML-DSA is a decision whose performance cost is smaller than the cost of not migrating.

## References

- [1] Amazon Web Services. Validating CloudTrail log file integrity, 2023. URL <https://docs.aws.amazon.com/awscloudtrail/latest/userguide/cloudtrail-log-file-validation-intro.html>.
- [2] Amazon Web Services. Using server-side encryption with AWS KMS keys (SSE-KMS), 2024. URL <https://docs.aws.amazon.com/AmazonS3/latest/userguide/UsingKMSEncryption.html>.
- [3] Amazon Web Services. Checking object integrity in Amazon S3, 2024. URL <https://docs.aws.amazon.com/AmazonS3/latest/userguide/checking-object-integrity.html>.
- [4] Amazon Web Services. Checking object integrity for data at rest in Amazon S3, 2024. URL <https://docs.aws.amazon.com/AmazonS3/latest/userguide/checking-object-integrity-at-rest.html>.
- [5] Amazon Web Services. Client-side encryption, 2024. URL <https://docs.aws.amazon.com/kms/latest/cryptographic-details/client-side-encryption.html>.
- [6] Elaine Barker and Allen Roginsky. Transitioning the use of cryptographic algorithms and key lengths, March 2019.
- [7] C. Bormann and P. Hoffman. Concise binary object representation (CBOR). RFC 8949, December 2020.
- [8] Wouter Castryck and Thomas Decru. An efficient key recovery attack on SIDH (preliminary version), July 2022. URL <https://eprint.iacr.org/2022/975>.
- [9] Google Cloud. Google cloud KMS now supports quantum-safe digital signatures, February 2025. URL <https://cloud.google.com/blog/products/identity-security/google-cloud-kms-now-supports-quantum-safe-digital-signatures>.
- [10] Pierre-Alain Fouque, Jeffrey Hoffstein, Paul Kirchner, Vadim Lyubashevsky, Thomas Pornin, Thomas Prest, Thomas Pulles, Christine van



- Vredendaal, William Whyte, and Zhenfei Zhang. Fn-dsa: Fast-Fourier lattice-based compact signatures over NTRU. NIST Post-Quantum Cryptography Round 3 Submission, 2020. URL [https://fn\\_dsa-sign.info](https://fn_dsa-sign.info).
- [11] Yanina García-Ruiz, Bernhard Teufel, Pantelis Stanitsas, et al. aws-s3-integrity-check: an open-source bash tool to verify the integrity of a dataset stored on Amazon S3. *GigaByte*, 2023. doi: 10.46471/gigabyte.87.
  - [12] Craig Gidney and Martin Ekerå. How to factor 2048 bit rsa integers in 8 hours using 20 million noisy qubits. *Quantum*, 5:433, 2021. doi: 10.22331/q-2021-04-15-433.
  - [13] Google Cloud. Envelope encryption, 2024. URL <https://cloud.google.com/kms/docs/envelope-encryption>.
  - [14] Thales Group. ML-DSA programming guide for luna HSM, November 2025. URL [https://www.thalesdocs.com/gphsm/luna/7/docs/network/Content/sdk/extensions/pqc/ML-DSA\\_programming\\_guide.htm](https://www.thalesdocs.com/gphsm/luna/7/docs/network/Content/sdk/extensions/pqc/ML-DSA_programming_guide.htm).
  - [15] Lov K. Grover. A fast quantum mechanical algorithm for database search. In *Proceedings of the 28th Annual ACM Symposium on Theory of Computing*, pages 212–219, 1996. doi: 10.1145/237814.237866.
  - [16] HashiCorp. Harvest now, decrypt later: Why today’s encrypted data isn’t safe forever, 2025. URL <https://www.hashicorp.com/blog/harvest-now-decrypt-later>.
  - [17] Jeffrey Hoffstein, Jill Pipher, and Joseph H. Silverman. NTRU: A ring-based public key cryptosystem. In *Algorithmic Number Theory – ANTS-III 1998*, volume 1423 of *Lecture Notes in Computer Science*, pages 267–288, Berlin, 1998. Springer. doi: 10.1007/BFb0054868.
  - [18] A. Huelsing, D. Butin, S. Gazdag, J. Rijneveld, and A. Mohaisen. XMSS: extended merkle signature scheme. RFC 8391, May 2018.
  - [19] Karen Kent and Murugiah Souppaya. Guide to computer security log management. NIST Special Publication 800-92, September 2006. URL <https://csrc.nist.gov/pubs/sp/800/92/final>.
  - [20] Leslie Lamport. Constructing digital signatures from a one-way function. Technical Report SRI-CSL-98, SRI International Computer Science Laboratory, 1979.

- [21] Vadim Lyubashevsky. Lattice signatures without trapdoors. In *Advances in Cryptology – EUROCRYPT 2012*, volume 7237 of *Lecture Notes in Computer Science*, pages 738–755, Berlin, 2012. Springer. doi: 10.1007/978-3-642-29011-4\_43.
- [22] J. Massimo, P. Kampanakis, S. Turner, and B. Westerbaan. Internet X.509 public key infrastructure: Algorithm identifiers for ML-DSA. RFC 9881, October 2025.
- [23] J. Massimo et al. Internet X.509 public key infrastructure: Algorithm identifiers for Kyber, 2024. URL <https://datatracker.ietf.org/doc/draft-ietf-lamps-kyber-certificates/>. IETF Internet-Draft.
- [24] D. McGrew, M. Curcio, and S. Fluhrer. Leighton-micali hash-based signatures. RFC 8554, April 2019.
- [25] Ralph C. Merkle. A certified digital signature. In *Advances in Cryptology – CRYPTO 1989*, volume 435 of *Lecture Notes in Computer Science*, pages 218–238, Berlin, 1989. Springer. doi: 10.1007/0-387-34805-0\_21.
- [26] MinIO, Inc. MinIO object storage for Linux, 2024. URL <https://min.io/docs/minio/linux/index.html>.
- [27] Michele Mosca. Cybersecurity in an era with quantum computers: Will we be ready? *IEEE Security & Privacy*, 16(5):38–41, 2018. doi: 10.1109/MSP.2018.3761723.
- [28] National Institute of Standards and Technology. SHA-3 standard: Permutation-based hash and extendable-output functions. FIPS PUB 202, August 2015.
- [29] Jack O’Connor, Jean-Philippe Aumasson, Samuel Neves, and Zooko Wilcox-O’Hearn. BLAKE3: One function, fast everywhere. In *IACR Cryptology ePrint Archive*, 2020. URL <https://eprint.iacr.org/2020/1235>. Report 2020/1235.
- [30] National Cybersecurity Center of Excellence. Migration to post-quantum cryptography, 2023. URL <https://www.nccoe.nist.gov/crypto-agility-considerations-migrating-post-quantum-cryptographic-algorithms>.
- [31] National Institute of Standards and Technology. Announcing request for nominations for public-key post-quantum cryptographic algorithms, December 2016. URL <https://csrc.nist.gov/news/2016/public-key-post-quantum-cryptographic-algorithms>. Federal Register Notice.

- [32] National Institute of Standards and Technology. Post-quantum cryptography: Round 2 submissions, January 2019. URL <https://csrc.nist.gov/projects/post-quantum-cryptography/round-2-submissions>.
- [33] National Institute of Standards and Technology. Post-quantum cryptography: Third round candidates, July 2020. URL <https://csrc.nist.gov/projects/post-quantum-cryptography/round-3-submissions>.
- [34] National Institute of Standards and Technology. Recommendation for stateful hash-based signature schemes, October 2020.
- [35] National Institute of Standards and Technology. Digital signature standard (dss), February 2023.
- [36] National Institute of Standards and Technology. Module-lattice-based key encapsulation mechanism standard, August 2024.
- [37] National Institute of Standards and Technology. Module-lattice-based digital signature standard, August 2024.
- [38] National Institute of Standards and Technology. Stateless hash-based digital signature standard, August 2024.
- [39] National Institute of Standards and Technology. FN-DSA: FN-DSA-based digital signature algorithm, August 2024.
- [40] National Institute of Standards and Technology. Transition to post-quantum cryptography standards, November 2024.
- [41] National Institute of Standards and Technology. Post-quantum cryptography: Workshops and timeline, 2024. URL <https://csrc.nist.gov/projects/post-quantum-cryptography/workshops-and-timeline>.
- [42] National Institute of Standards and Technology. NIST selects HQC as fifth algorithm for post-quantum encryption, March 2025. URL <https://www.nist.gov/news-events/news/2025/03/nist-selects-hqc-fifth-algorithm-post-quantum-encryption>.
- [43] National Institute of Standards and Technology. Status report on the fourth round of the nist post-quantum cryptography standardization process, March 2025.
- [44] John Preskill. Quantum computing in the NISQ era and beyond. *Quantum*, 2:79, 2018. doi: 10.22331/q-2018-08-06-79.

- [45] Oded Regev. On lattices, learning with errors, random linear codes, and cryptography. *Journal of the ACM*, 56(6):34:1–34:40, 2009. doi: 10.1145/1568318.1568324.
- [46] Sectigo. Harvest now, decrypt later: Quantum threat, 2025. URL <https://www.sectigo.com/resource-library/harvest-now-decrypt-later-quantum-threat>.
- [47] Amazon Web Services. ML-KEM post-quantum TLS now supported in AWS KMS, ACM, and secrets manager, April 2025. URL <https://aws.amazon.com/blogs/security/ml-kem-post-quantum-tls-now-supported-in-aws-kms-acm-and-secrets-manager/>.
- [48] Peter W. Shor. Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. *SIAM Journal on Computing*, 26(5):1484–1509, 1997.
- [49] Yandex Cloud. Envelope encryption, 2024. URL <https://yandex.cloud/en/docs/kms/concepts/envelope>.

# Appendices

## A Supplementary Submission Material

This appendix block is reserved for formal review outputs, project-management evidence, and any additional artefacts required by the final submission regulations. In the final submission package, this section should contain the second-marker review form, diary sheets, and any supplementary design or user documentation included as appendices.